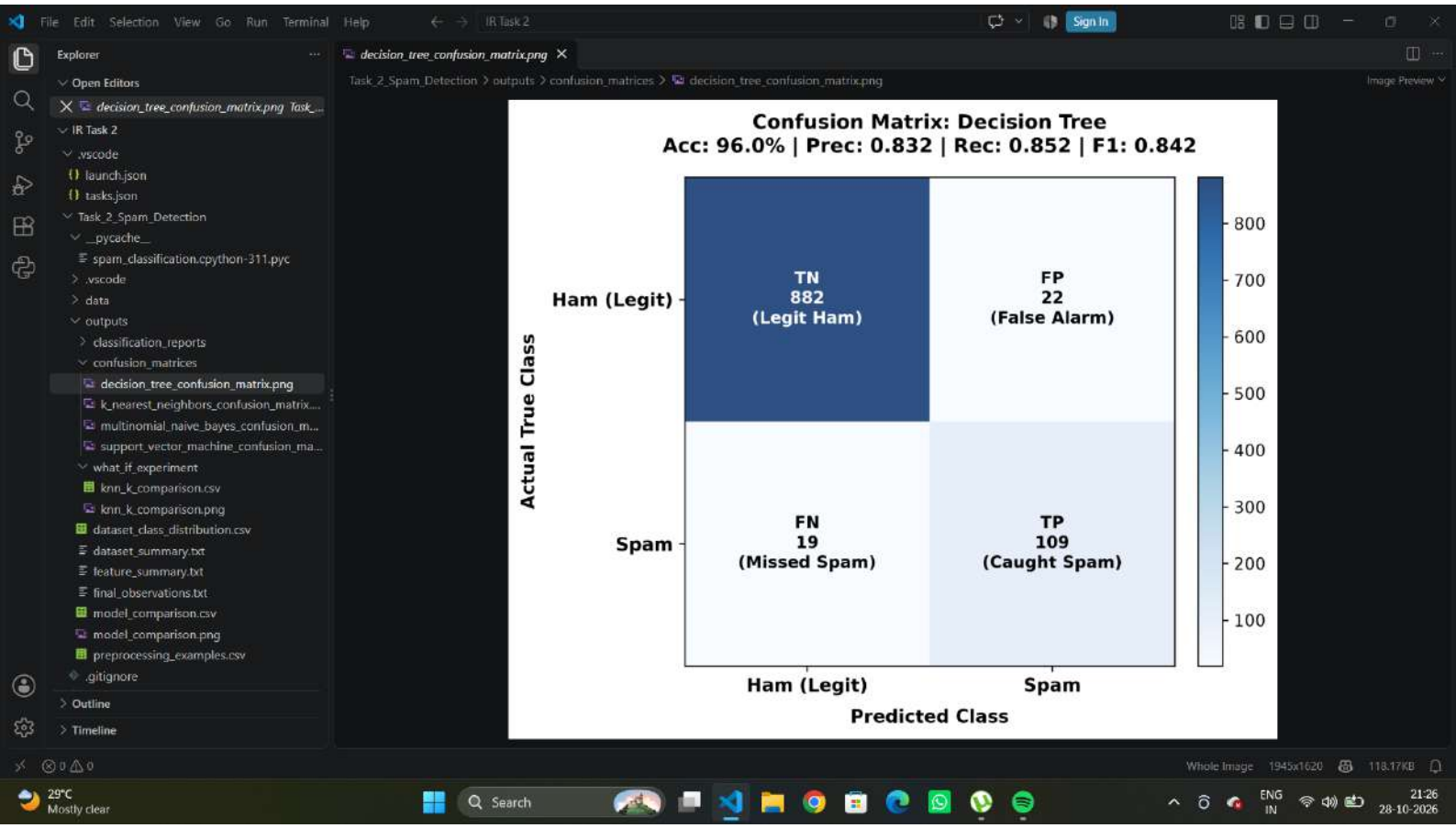
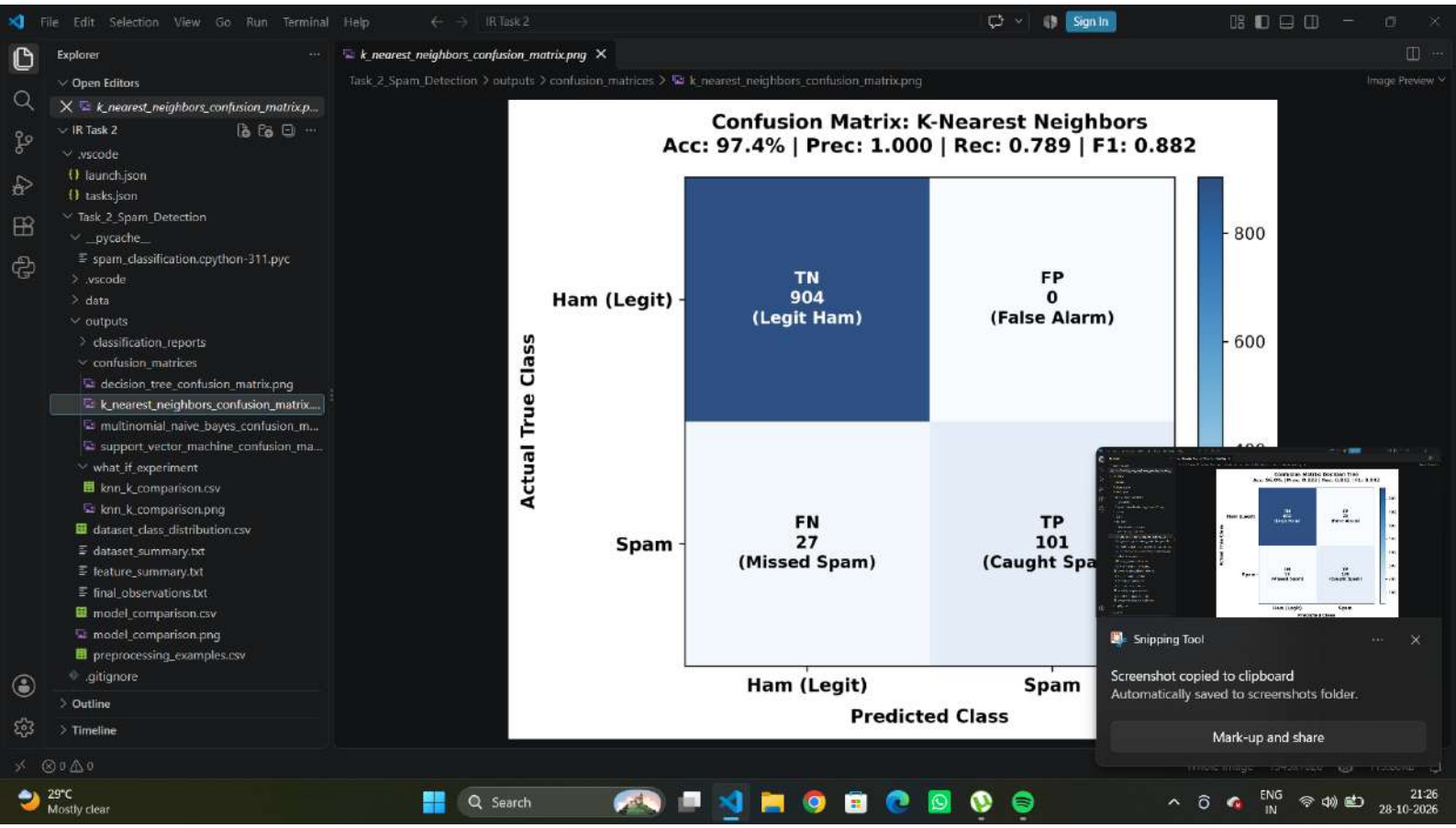
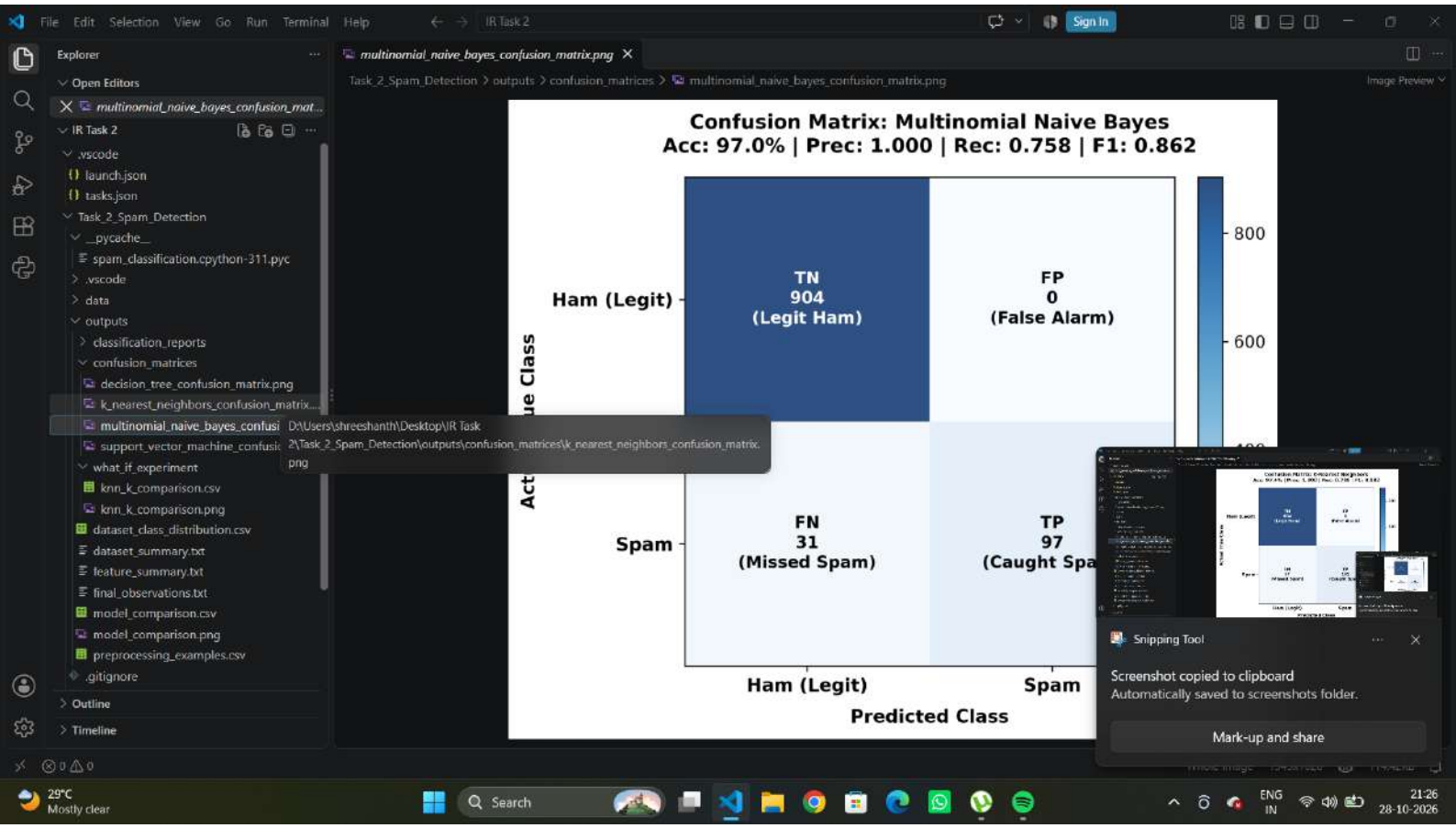
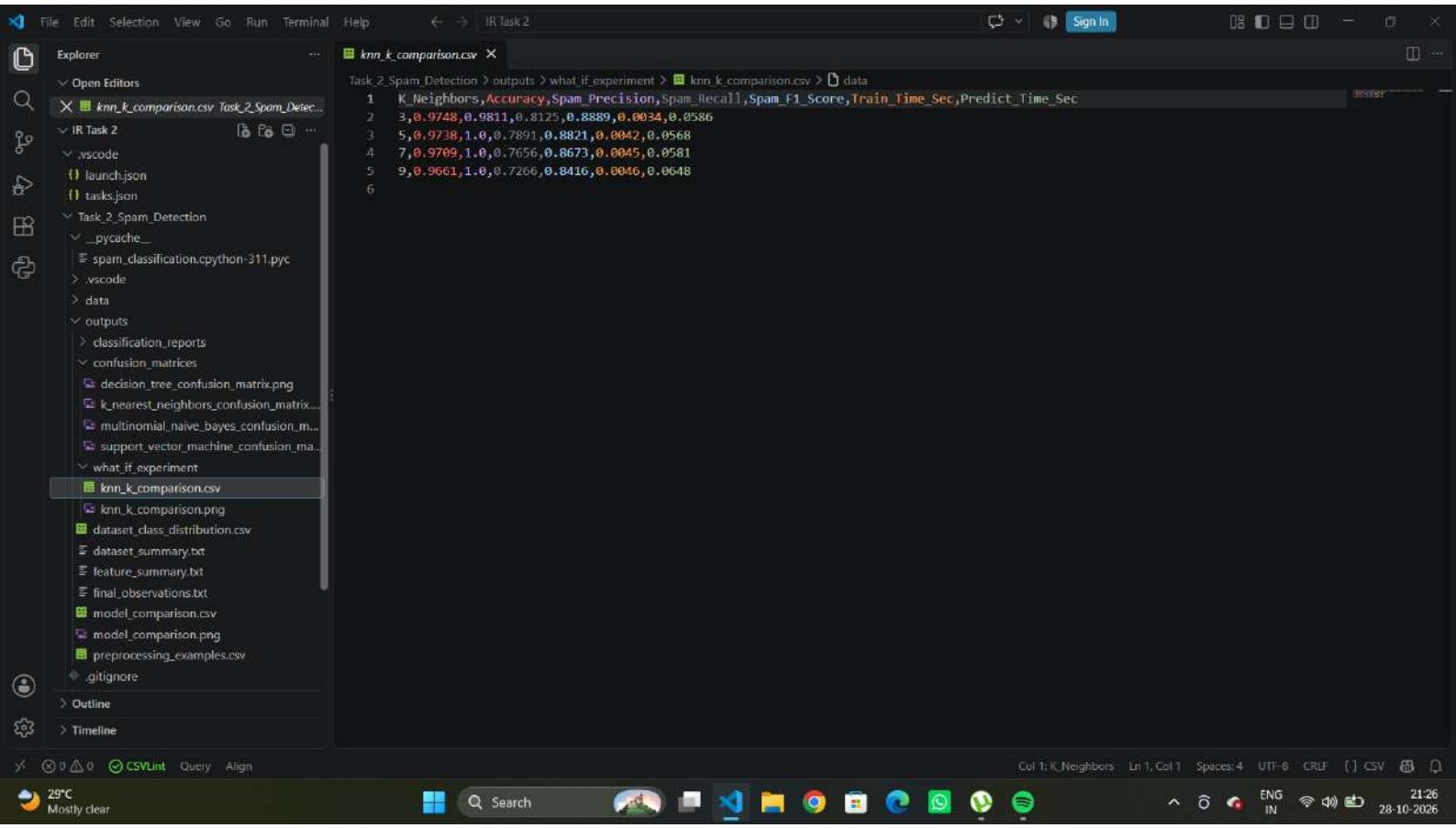
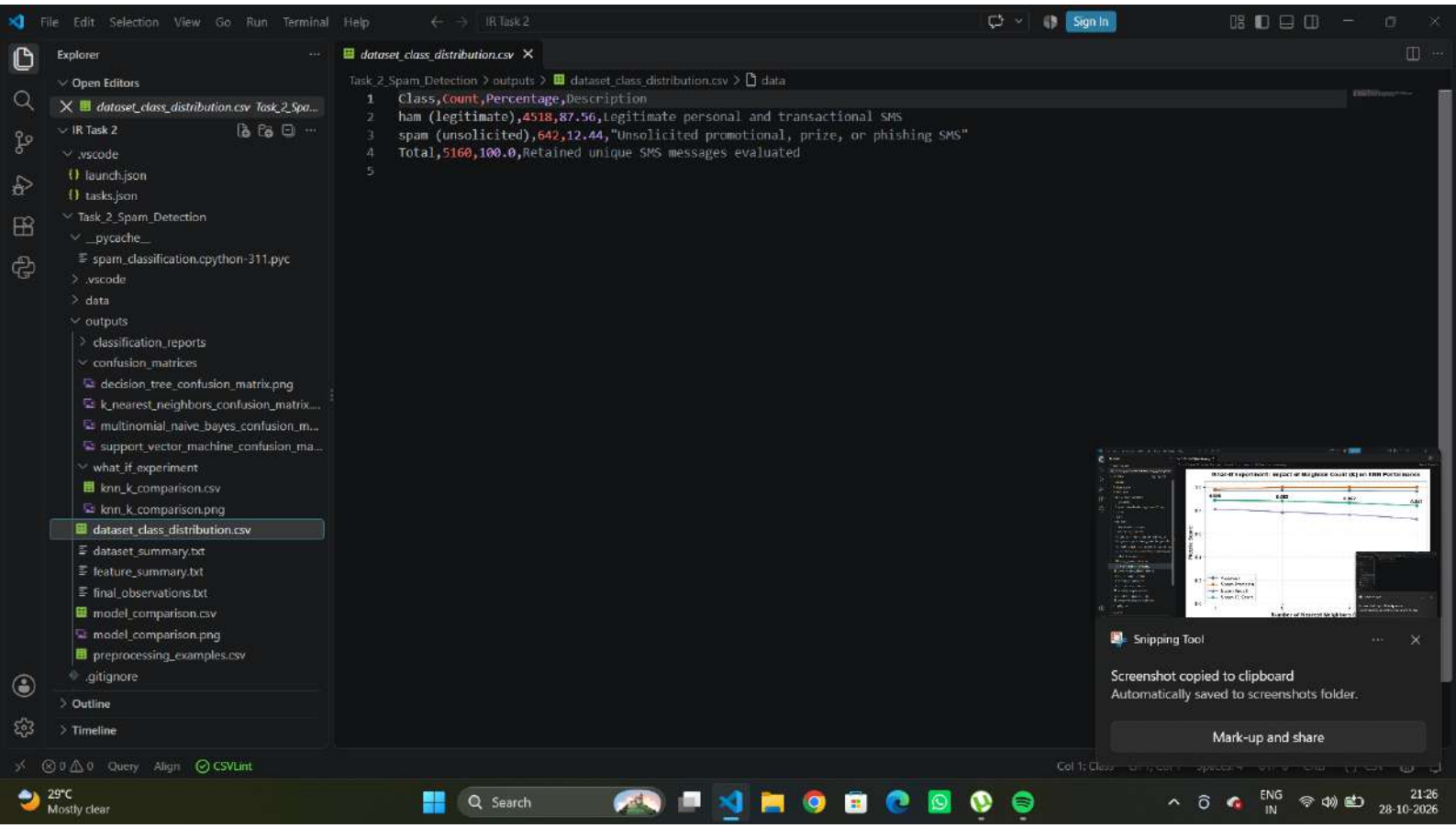

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
decision_tree_confusion_matrix.png Task_...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
decision_tree_confusion_matrix.png
Task_2_Spam_Detection > outputs > confusion_matrices > decision_tree_confusion_matrix.png
Image Preview
Confusion Matrix: Decision Tree
Acc: 96.0% | Prec: 0.832 | Rec: 0.852 | F1: 0.842
TN
882
(Legit Ham)
FP
22
(False Alarm)
FN
19
(Missed Spam)
TP
109
(Caught Spam)
Ham (Legit)
Spam
Actual True Class
Ham (Legit)
Spam
Predicted Class
800
700
600
500
400
300
200
100
Whole Image 1945x1620 118.17KB
29°C
Mostly clear
Search
ENG
IN
21:26
28-10-2026

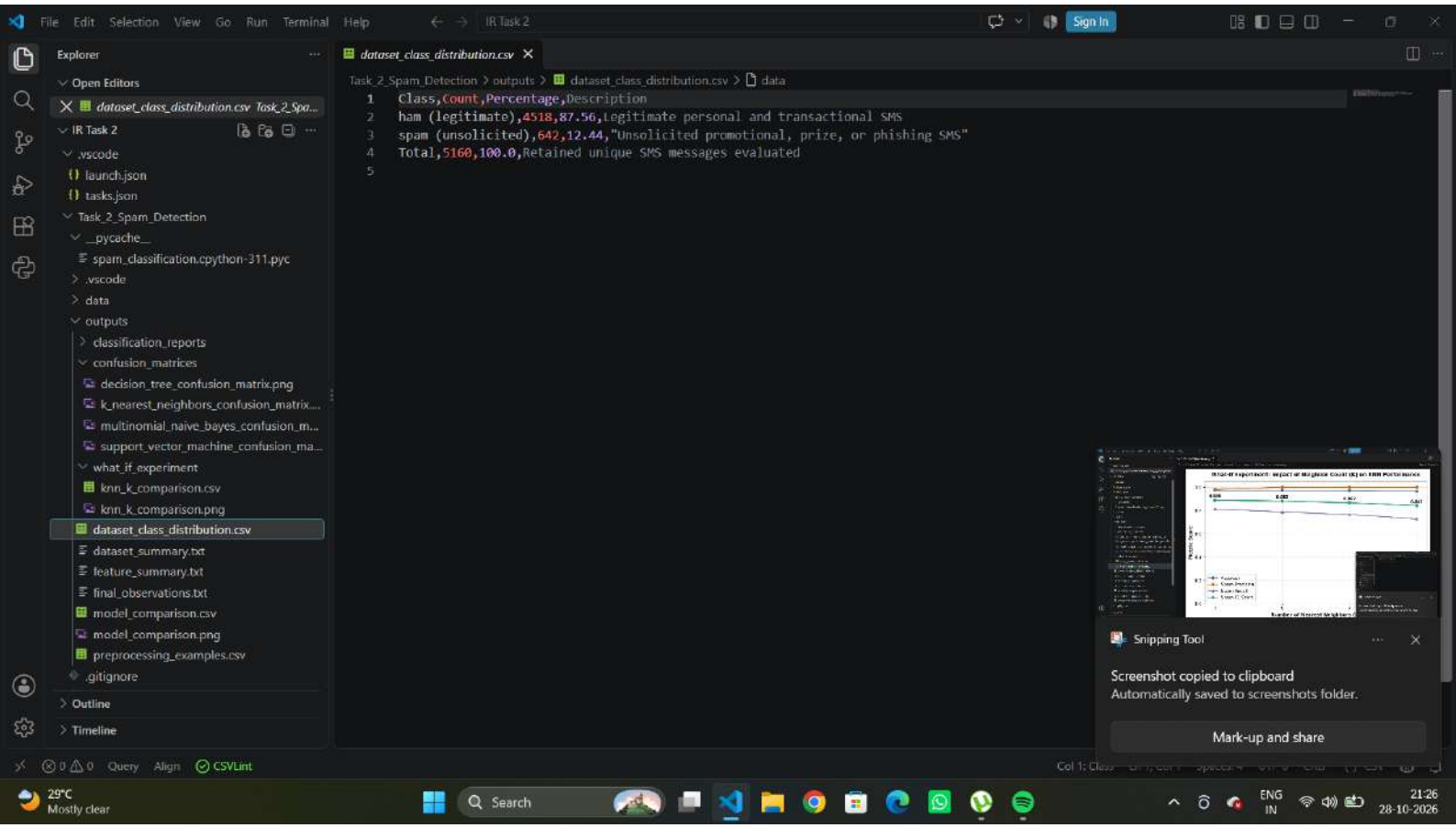

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
k_nearest_neighbors_confusion_matrix.p...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
k_nearest_neighbors_confusion_matrix.png
Task_2_Spam_Detection > outputs > confusion_matrices > k_nearest_neighbors_confusion_matrix.png
Image Preview
Confusion Matrix: K-Nearest Neighbors
Acc: 97.4% | Prec: 1.000 | Rec: 0.789 | F1: 0.882
TN
904
(Legit Ham)
FP
0
(False Alarm)
FN
27
(Missed Spam)
TP
101
(Caught Spa
Ham (Legit)
Spam
Actual True Class
Ham (Legit)
Spam
Predicted Class
800
600
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
29°C
Mostly clear
Search
ENG
IN
21:26
28-10-2026

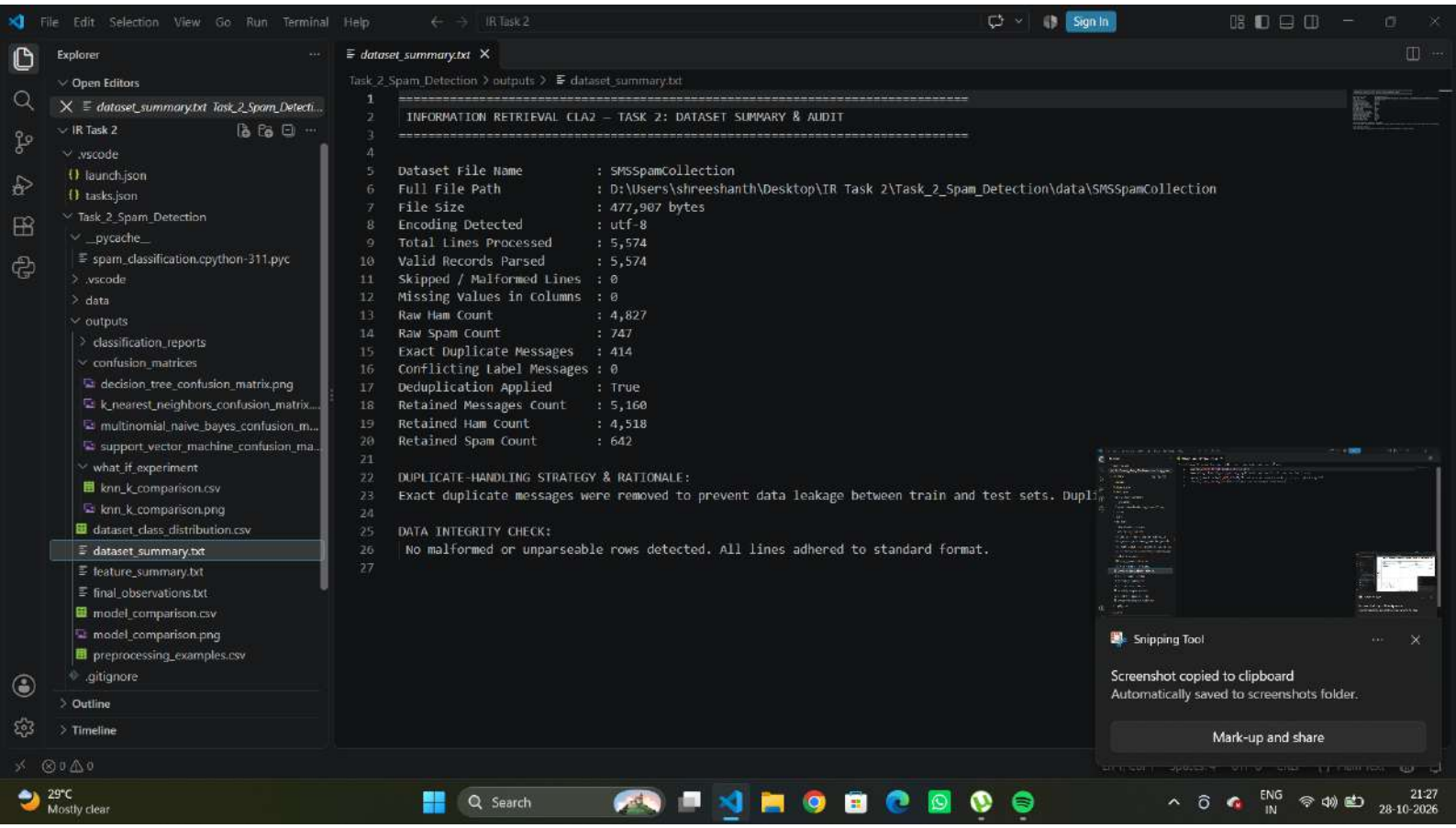

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
multinomial_naive_bayes_confusion_mat...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusi
support_vector_machine_confusi
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
multinomial_naive_bayes_confusion_matrix.png
Task_2_Spam_Detection > outputs > confusion_matrices > multinomial_naive_bayes_confusion_matrix.png
Image Preview
D:\Users\shreeshanth\Desktop\IR Task 2\Task_2_Spam_Detection\outputs\confusion_matrices\k_nearest_neighbors_confusion_matrix.png
Confusion Matrix: Multinomial Naive Bayes
Acc: 97.0% | Prec: 1.000 | Rec: 0.758 | F1: 0.862
TN
904
(Legit Ham)
FP
0
(False Alarm)
FN
31
(Missed Spam)
TP
97
(Caught Spa
Ham (Legit)
Spam
Actual Class
Ham (Legit)
Spam
Predicted Class
800
600
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
29°C
Mostly clear
Search
ENG
IN
21:26
28-10-2026

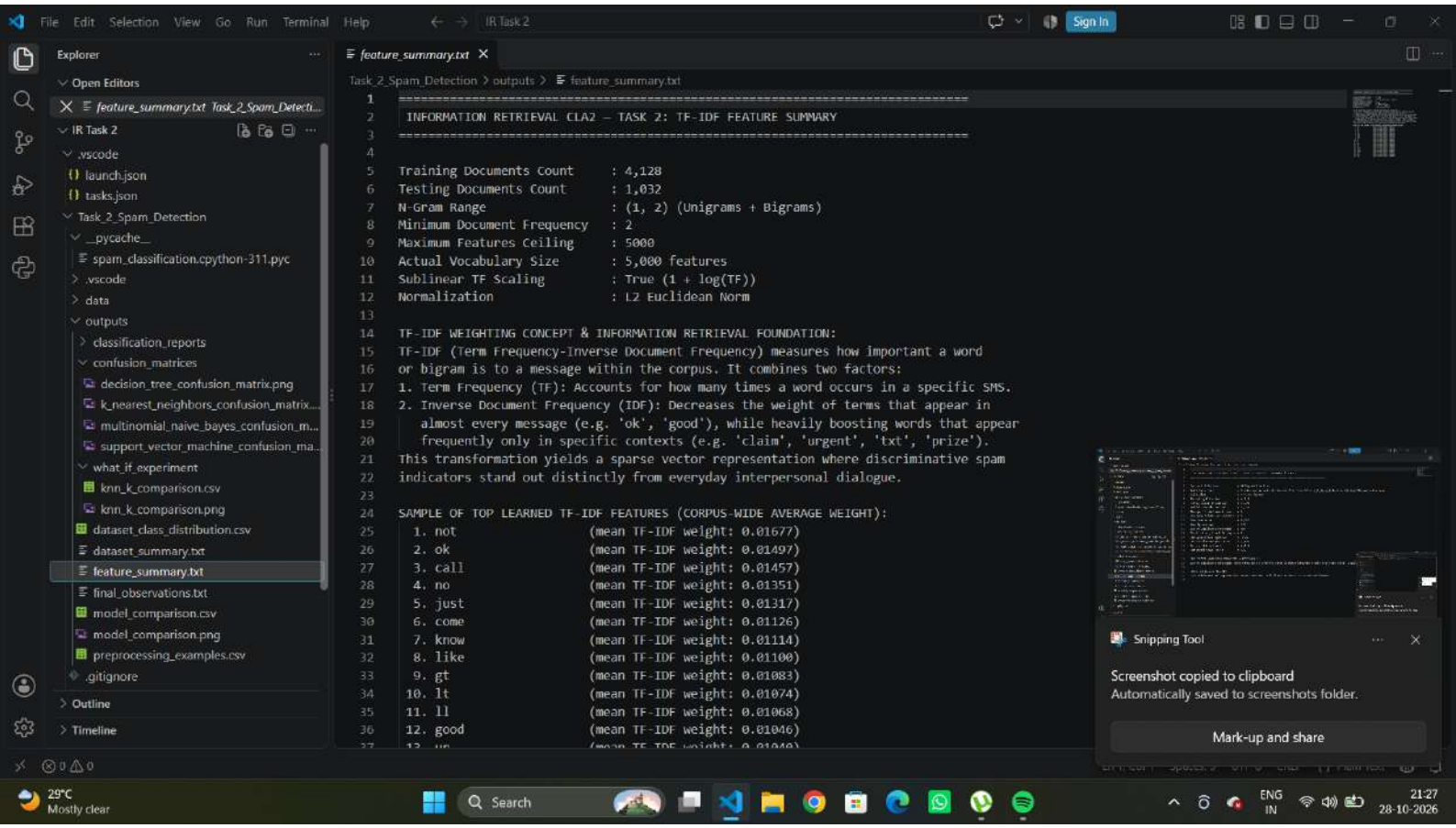

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
knn_k_comparison.csv Task_2_Spam_Detec...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix...
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
knn_k_comparison.csv
Task_2_Spam_Detection > outputs > what_if_experiment > knn_k_comparison.csv > data
1 K_Neighbors,Accuracy,Spam_Precision,Spam_Recall,Spam_F1_Score,Train_Time_Sec,Predict_Time_Sec
2 3,0.9748,0.9811,0.8125,0.8889,0.0034,0.0586
3 5,0.9738,1.0,0.7891,0.8821,0.0042,0.0568
4 7,0.9709,1.0,0.7656,0.8673,0.0045,0.0581
5 9,0.9661,1.0,0.7266,0.8416,0.0046,0.0648
6
CSVLint
Query
Align
Col 1: K_Neighbors
Ln 1, Col 1
Spaces: 4
UTF-8
CRLF
CSV
29°C
Mostly clear
Search
ENG
IN
21:26
28-10-2026

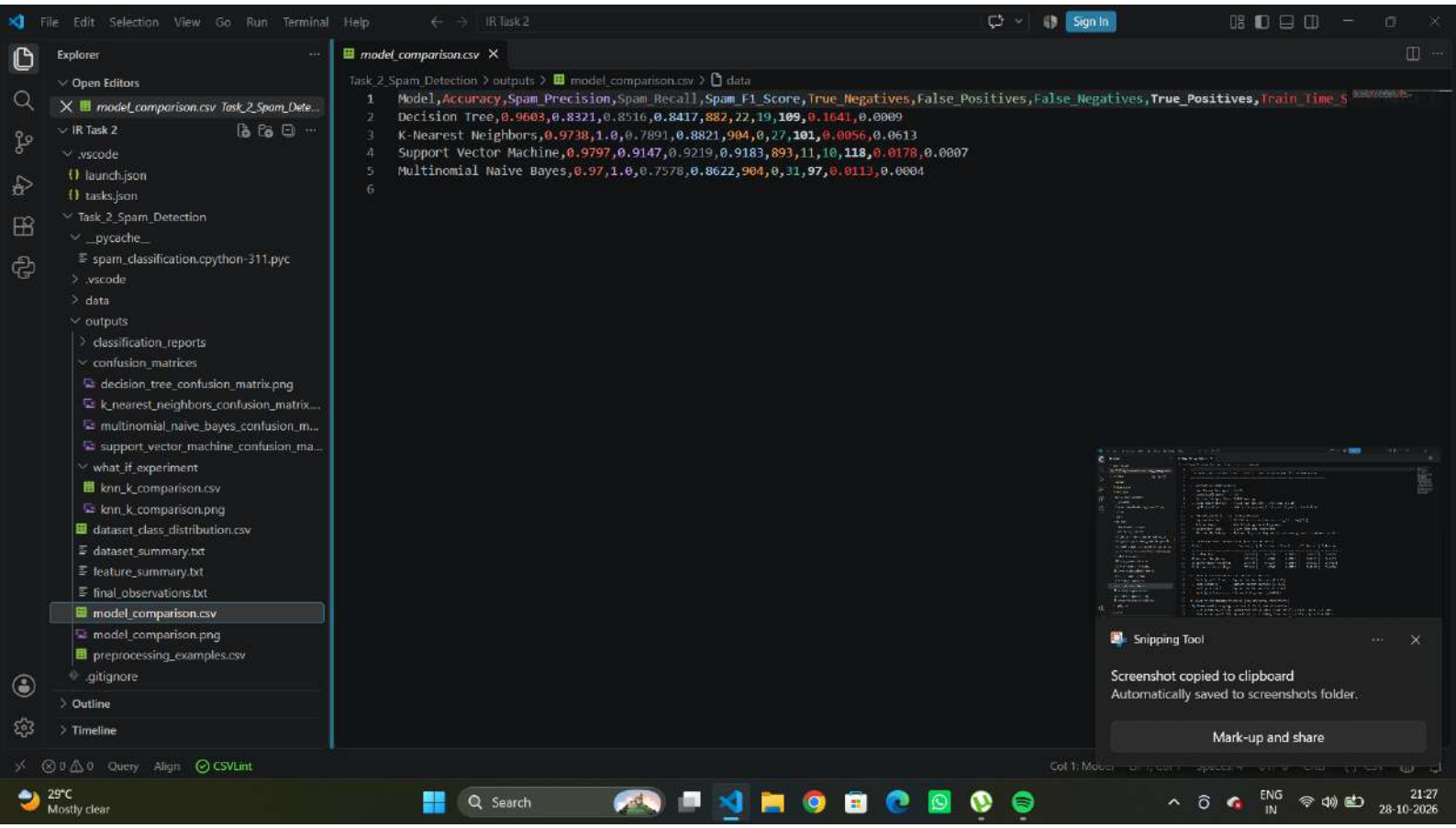

What-If Experiment: Impact of Neighbor Count (K) on KNN Performance
Metric Score
Number of Nearest Neighbors (
Accuracy
Spam Precision
Spam Recall
Spam F1-Score
0.889
0.882
0.867
0.842
1.0
0.8
0.6
0.4
0.2
0.0
3
5
7

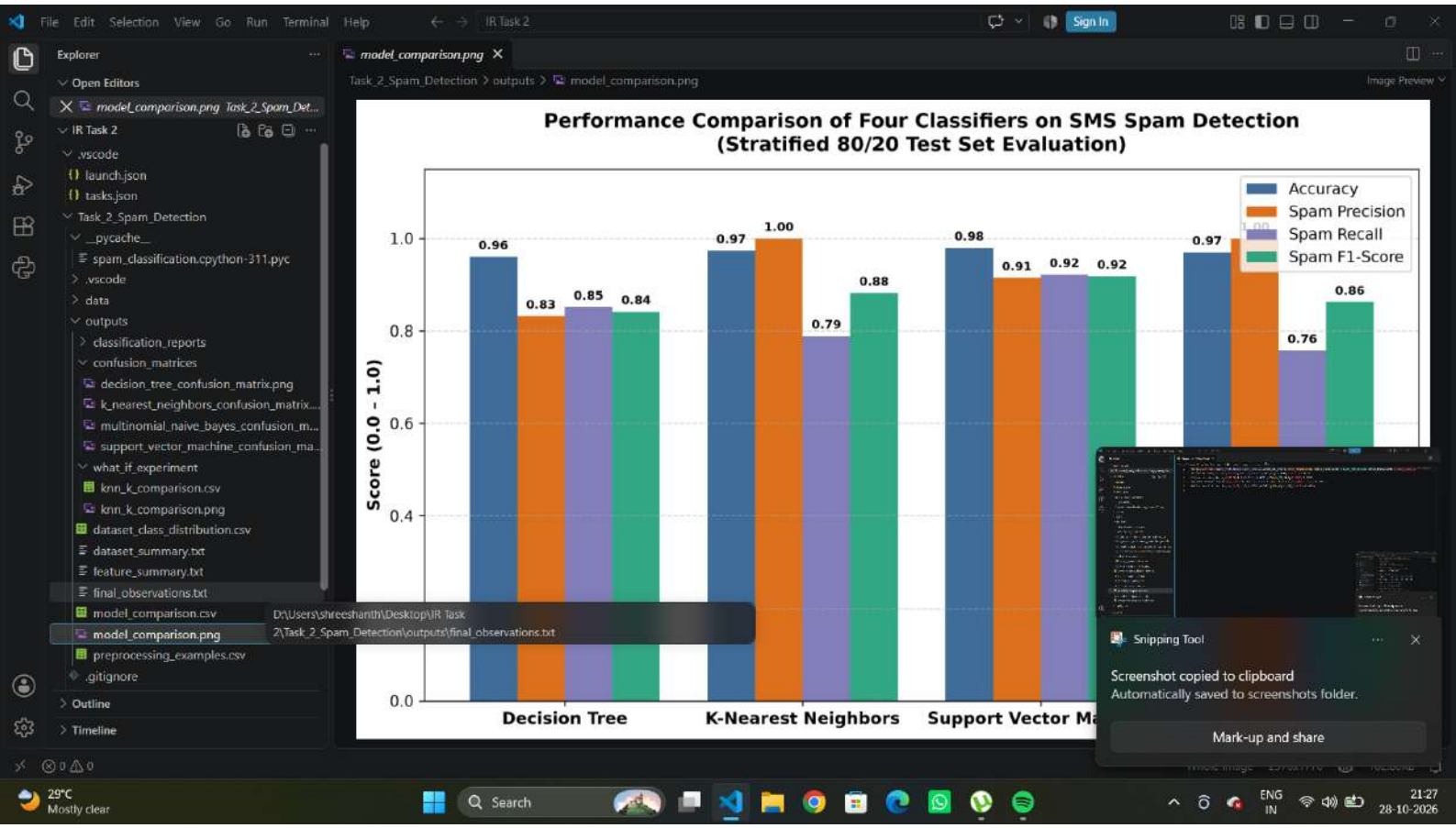

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
dataset_class_distribution.csv Task_2_Spa...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
dataset_class_distribution.csv
Task_2_Spam_Detection > outputs > dataset_class_distribution.csv > data
1 Class,Count,Percentage,Description
2 ham (legitimate),4518,87.56,Legitimate personal and transactional SMS
3 spam (unsolicited),642,12.44,"Unsolicited promotional, prize, or phishing SMS"
4 Total,5160,100.0,Retained unique SMS messages evaluated
5
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Query
Align
CSVLint
29°C
Mostly clear
Search
ENG
IN
21:26
28-10-2026

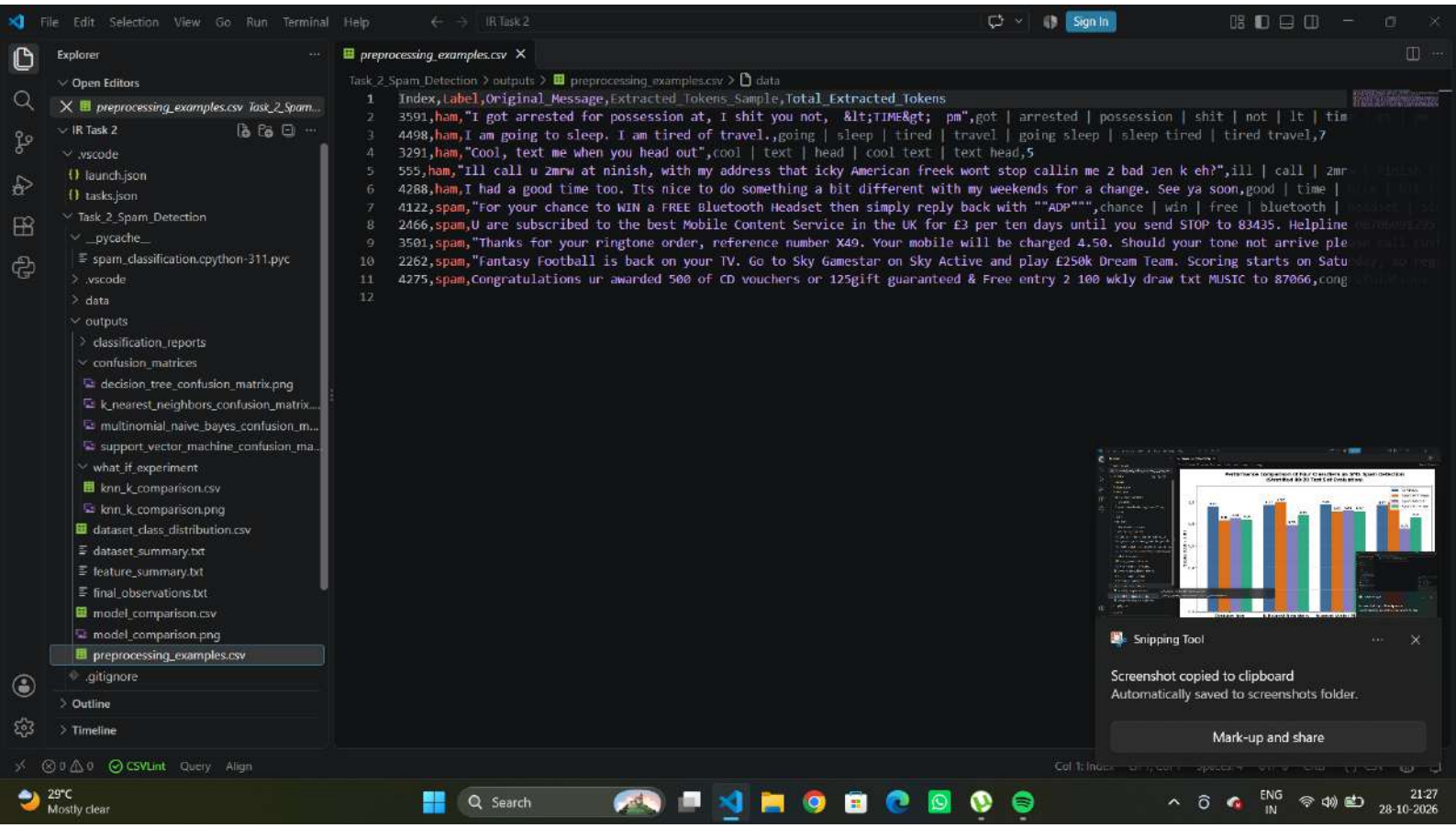

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
dataset_class_distribution.csv Task_2_Spa...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
dataset_class_distribution.csv
Task_2_Spam_Detection > outputs > dataset_class_distribution.csv > data
1 Class,Count,Percentage,Description
2 ham (legitimate),4518,87.56,Legitimate personal and transactional SMS
3 spam (unsolicited),642,12.44,"Unsolicited promotional, prize, or phishing SMS"
4 Total,5160,100.0,Retained unique SMS messages evaluated
5
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Query
Align
CSVLint
29°C
Mostly clear
Search
ENG
IN
21:26
28-10-2026

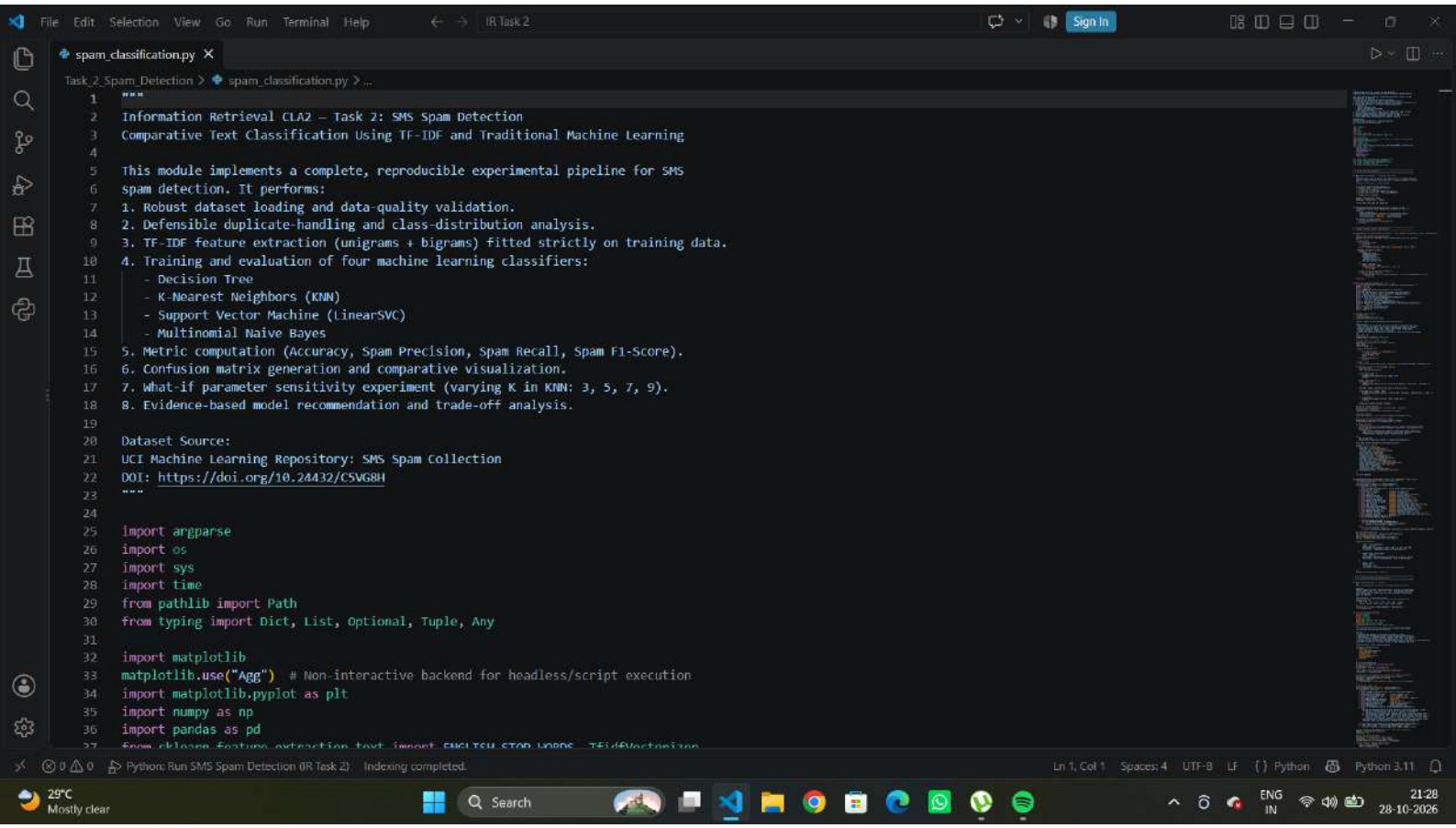

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
dataset_summary.txt Task_2_Spam_Detecti...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix...
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
dataset_summary.txt
Task_2_Spam_Detection > outputs > dataset_summary.txt
INFORMATION RETRIEVAL CLA2 – TASK 2: DATASET SUMMARY & AUDIT
Dataset File Name : SMSSpamCollection
Full File Path : D:\Users\shreeshanth\Desktop\IR Task 2\Task_2_Spam_Detection\data\SMSSpamCollection
File Size : 477,907 bytes
Encoding Detected : utf-8
Total Lines Processed : 5,574
Valid Records Parsed : 5,574
Skipped / Malformed Lines : 0
Missing Values in Columns : 0
Raw Ham Count : 4,827
Raw Spam Count : 747
Exact Duplicate Messages : 414
Conflicting Label Messages : 0
Deduplication Applied : True
Retained Messages Count : 5,160
Retained Ham Count : 4,518
Retained Spam Count : 642
DUPLICATE-HANDLING STRATEGY & RATIONALE:
Exact duplicate messages were removed to prevent data leakage between train and test sets. Dupl
DATA INTEGRITY CHECK:
No malformed or unparseable rows detected. All lines adhered to standard format.
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
29°C
Mostly clear
Search
ENG
IN
21:27
28-10-2026

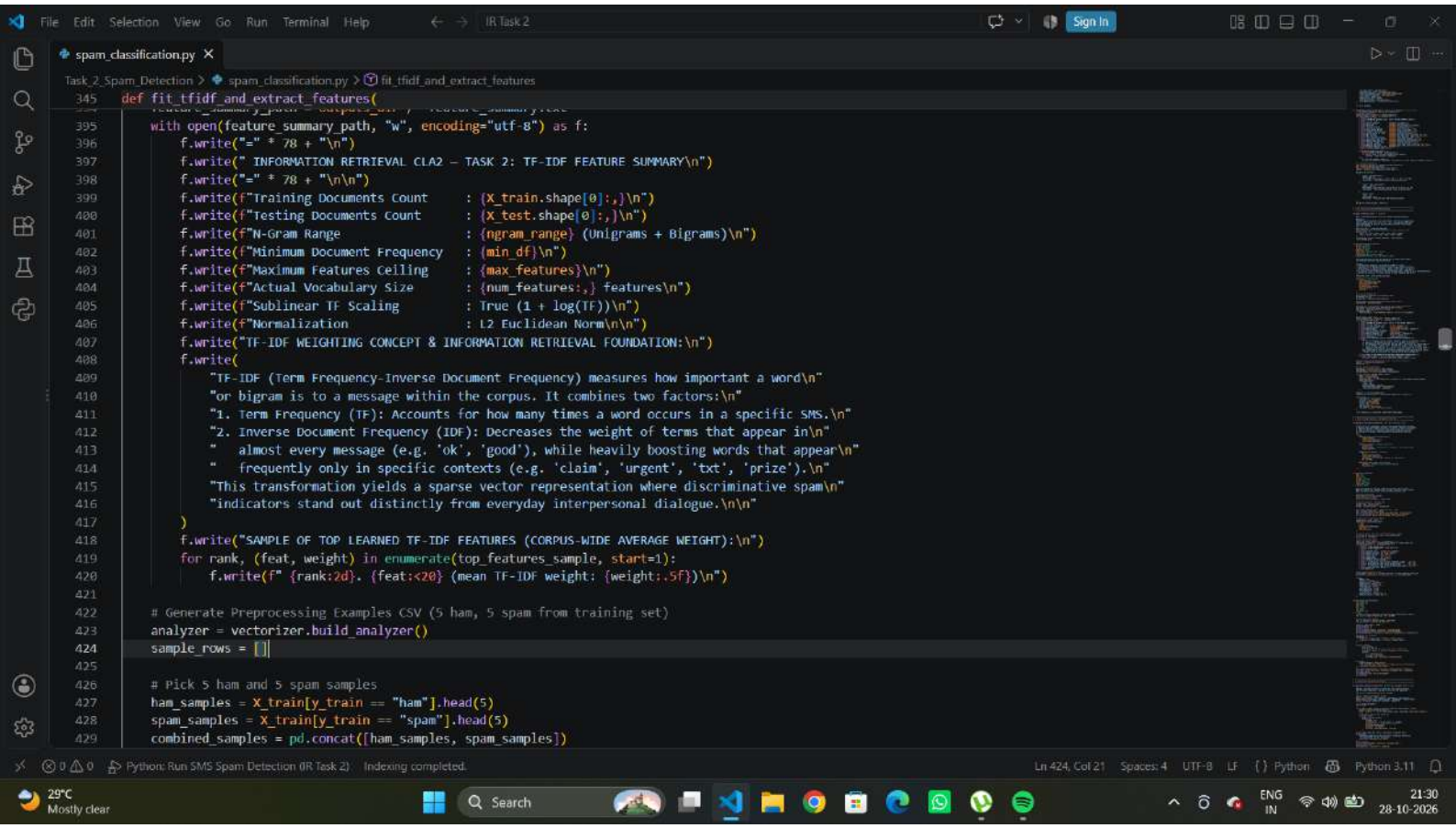

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
feature_summary.txt Task_2_Spam_Detecti...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
feature_summary.txt
Task_2_Spam_Detection > outputs > feature_summary.txt
INFORMATION RETRIEVAL CLA2 – TASK 2: TF-IDF FEATURE SUMMARY
Training Documents Count : 4,128
Testing Documents Count : 1,032
N-Gram Range : (1, 2) (Unigrams + Bigrams)
Minimum Document Frequency : 2
Maximum Features Ceiling : 5000
Actual Vocabulary Size : 5,000 features
Sublinear TF Scaling : True (1 + log(TF))
Normalization : L2 Euclidean Norm
TF-IDF WEIGHTING CONCEPT & INFORMATION RETRIEVAL FOUNDATION:
TF-IDF (Term Frequency-Inverse Document Frequency) measures how important a word
or bigram is to a message within the corpus. It combines two factors:
1. Term Frequency (TF): Accounts for how many times a word occurs in a specific SMS.
2. Inverse Document Frequency (IDF): Decreases the weight of terms that appear in
almost every message (e.g. 'ok', 'good'), while heavily boosting words that appear
frequently only in specific contexts (e.g. 'claim', 'urgent', 'txt', 'prize').
This transformation yields a sparse vector representation where discriminative spam
indicators stand out distinctly from everyday interpersonal dialogue.
SAMPLE OF TOP LEARNED TF-IDF FEATURES (CORPUS-WIDE AVERAGE WEIGHT):
1. not (mean TF-IDF weight: 0.01677)
2. ok (mean TF-IDF weight: 0.01497)
3. call (mean TF-IDF weight: 0.01457)
4. no (mean TF-IDF weight: 0.01351)
5. just (mean TF-IDF weight: 0.01317)
6. come (mean TF-IDF weight: 0.01126)
7. know (mean TF-IDF weight: 0.01114)
8. like (mean TF-IDF weight: 0.01100)
9. gt (mean TF-IDF weight: 0.01083)
10. lt (mean TF-IDF weight: 0.01074)
11. ll (mean TF-IDF weight: 0.01068)
12. good (mean TF-IDF weight: 0.01046)
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
29°C
Mostly clear
Search
ENG
IN
21:27
28-10-2026

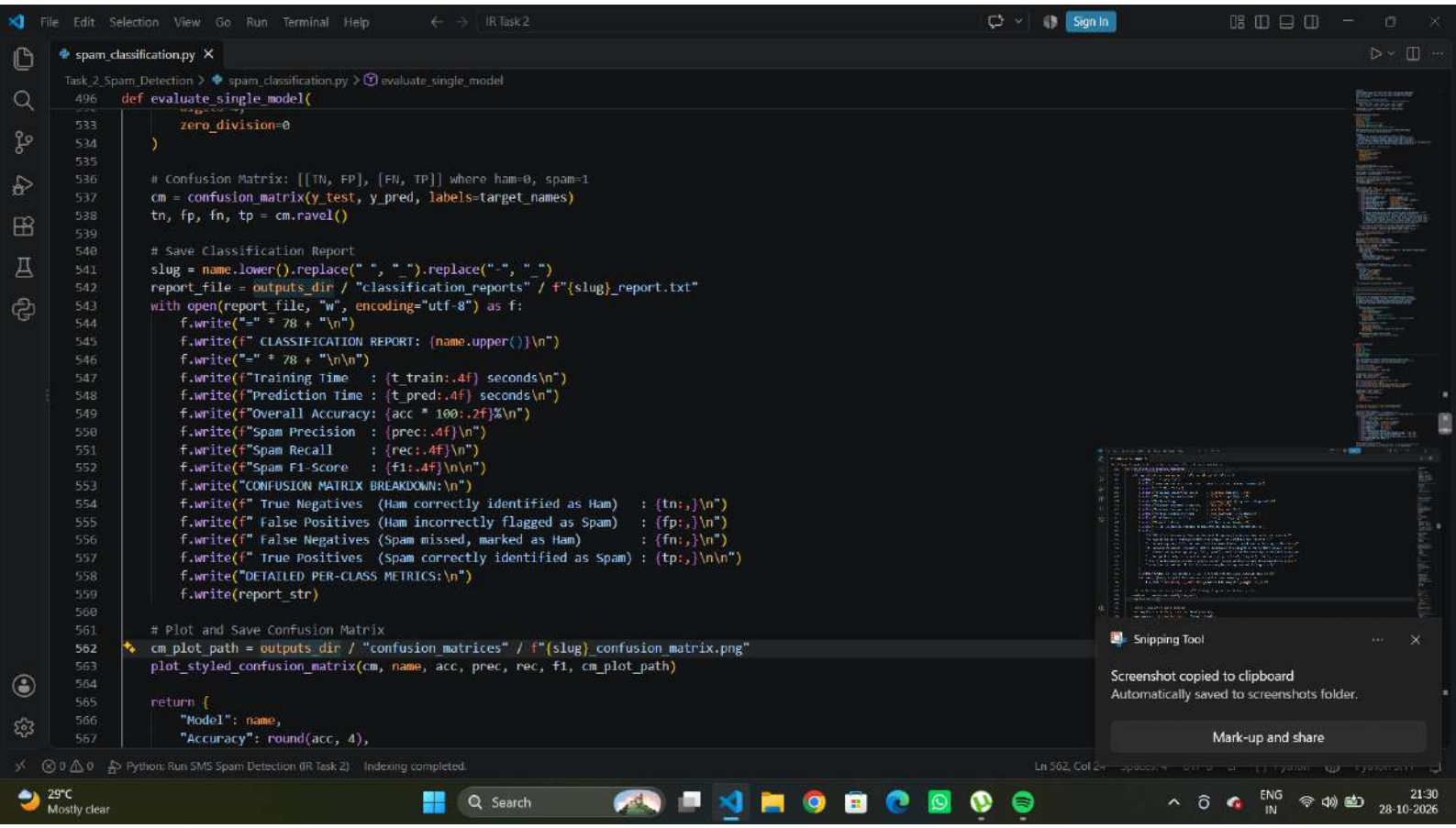

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
model_comparison.csv Task_2_Spam_Dete...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix....
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
model_comparison.csv
Task_2_Spam_Detection > outputs > model_comparison.csv > data
1 Model,Accuracy,Spam_Precision,Spam_Recall,Spam_F1_Score,True_Negatives,False_Positives,False_Negatives,True_Positives,Train_Time_S
2 Decision Tree,0.9603,0.8321,0.8516,0.8417,882,22,19,109,0.1641,0.0009
3 K-Nearest Neighbors,0.9738,1.0,0.7891,0.8821,904,0,27,101,0.0056,0.0613
4 Support Vector Machine,0.9797,0.9147,0.9219,0.9183,893,11,10,118,0.0178,0.0007
5 Multinomial Naive Bayes,0.97,1.0,0.7578,0.8622,904,0,31,97,0.0113,0.0004
6
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Query Align CSVLint
29°C
Mostly clear
Search
ENG
IN
21:27
28-10-2026

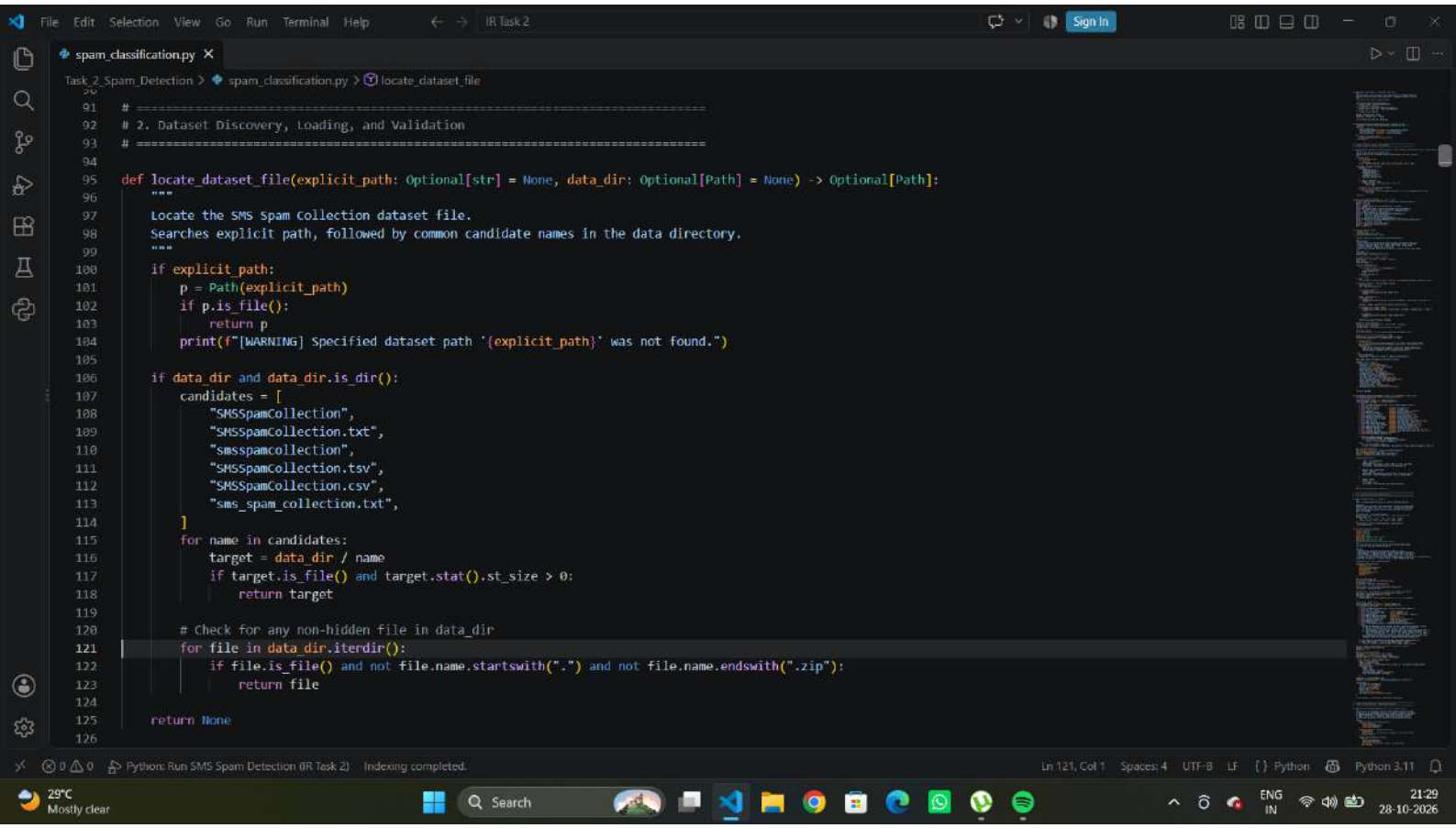

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
model_comparison.png Task_2_Spam_Det...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix...
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
model_comparison.png
Task_2_Spam_Detection > outputs > model_comparison.png
Image Preview
Performance Comparison of Four Classifiers on SMS Spam Detection
(Stratified 80/20 Test Set Evaluation)
Accuracy
Spam Precision
Spam Recall
Spam F1-Score
Score (0.0 - 1.0)
0.0
0.4
0.6
0.8
1.0
0.96
0.83
0.85
0.84
0.97
1.00
0.79
0.88
0.98
0.91
0.92
0.92
0.97
0.76
0.86
Decision Tree
K-Nearest Neighbors
Support Vector Ma
D:\Users\shreeshanth\Desktop\IR Task 2\Task_2_Spam_Detection\outputs\final_observations.txt
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
29°C
Mostly clear
Search
ENG
IN
21:27
28-10-2026

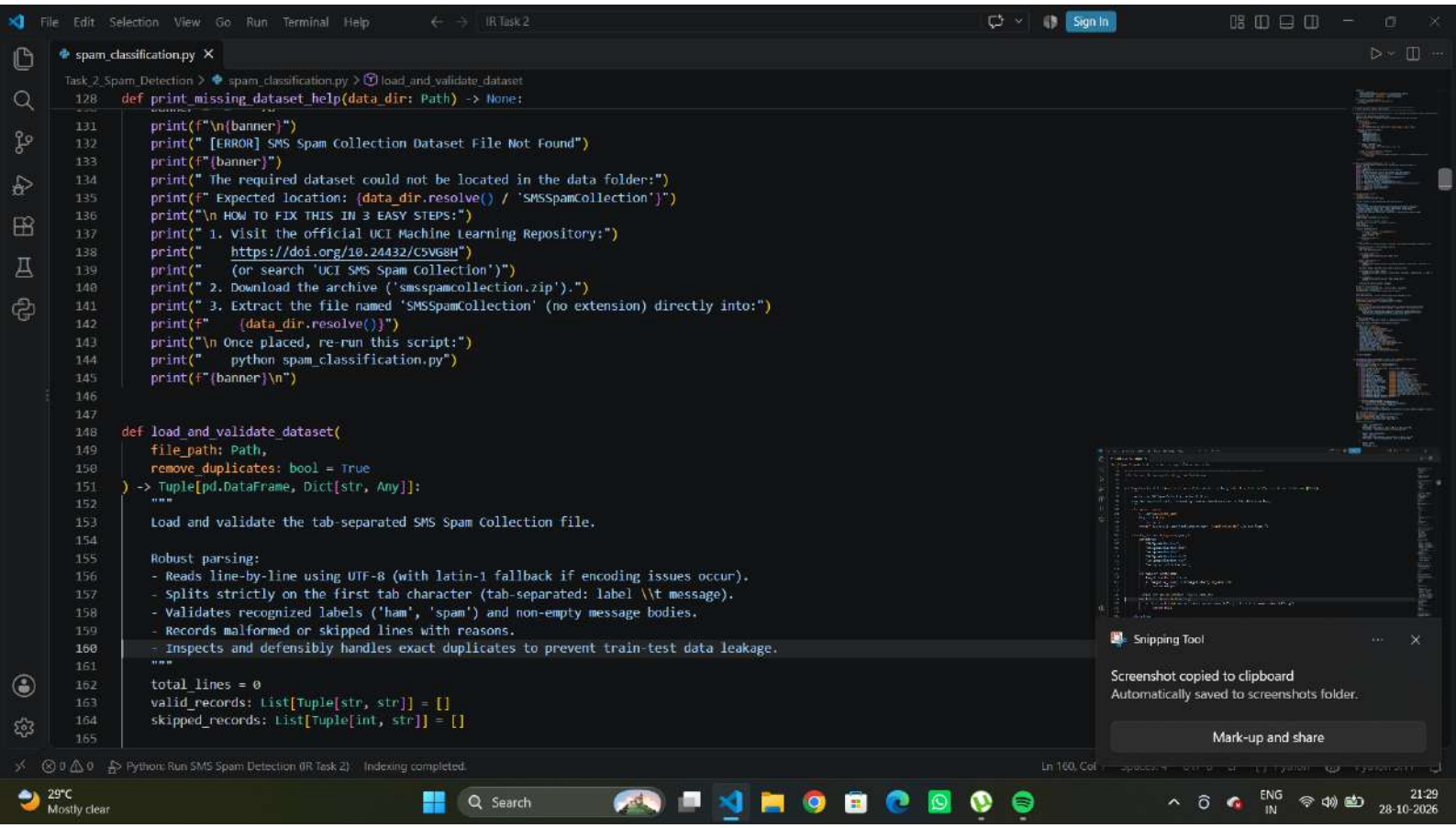

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
Explorer
Open Editors
preprocessing_examples.csv Task_2_Spam...
IR Task 2
.vscode
launch.json
tasks.json
Task_2_Spam_Detection
__pycache__
spam_classification.cpython-311.pyc
.vscode
data
outputs
classification_reports
confusion_matrices
decision_tree_confusion_matrix.png
k_nearest_neighbors_confusion_matrix...
multinomial_naive_bayes_confusion_m...
support_vector_machine_confusion_ma...
what_if_experiment
knn_k_comparison.csv
knn_k_comparison.png
dataset_class_distribution.csv
dataset_summary.txt
feature_summary.txt
final_observations.txt
model_comparison.csv
model_comparison.png
preprocessing_examples.csv
.gitignore
Outline
Timeline
preprocessing_examples.csv
Task_2_Spam_Detection > outputs > preprocessing_examples.csv > data
1 Index,Label,Original_Message,Extracted_Tokens_Sample,Total_Extracted_Tokens
2 3591,ham,"I got arrested for possession at, I shit you not, <TIME> pm",got | arrested | possession | shit | not | lt | tim
3 4498,ham,I am going to sleep. I am tired of travel.,going | sleep | tired | travel | going sleep | sleep tired | tired travel,7
4 3291,ham,"Cool, text me when you head out",cool | text | head | cool text | text head,5
5 555,ham,"Ill call u 2mrw at ninish, with my address that icky American freek wont stop callin me 2 bad Jen k eh?",ill | call | 2mr
6 4288,ham,I had a good time too. Its nice to do something a bit different with my weekends for a change. See ya soon,good | time |
7 4122,spam,"For your chance to WIN a FREE Bluetooth Headset then simply reply back with ""ADP""",chance | win | free | bluetooth |
8 2466,spam,U are subscribed to the best Mobile Content Service in the UK for £3 per ten days until you send STOP to 83435. Helpline
9 3501,spam,"Thanks for your ringtone order, reference number X49. Your mobile will be charged 4.50. Should your tone not arrive ple
10 2262,spam,"Fantasy Football is back on your TV. Go to Sky Gamestar on Sky Active and play £250k Dream Team. Scoring starts on Satu
11 4275,spam,Congratulations ur awarded 500 of CD vouchers or 125gift guaranteed & Free entry 2 100 wkly draw txt MUSIC to 87066,cong
12
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
CSVLint Query Align
29°C
Mostly clear
Search
ENG
IN
21:27
28-10-2026

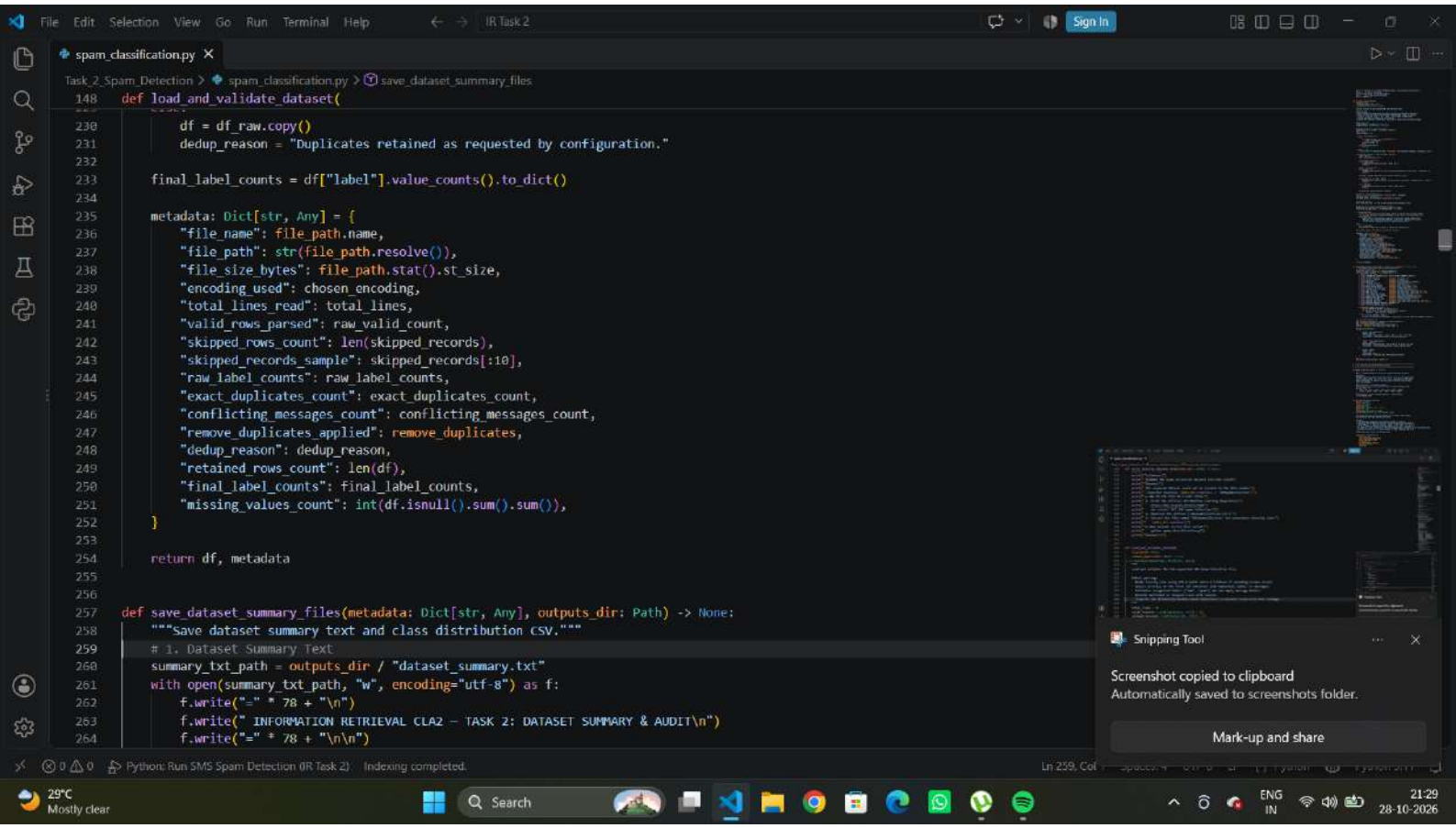

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > ...
"""
Information Retrieval CLA2 – Task 2: SMS Spam Detection
Comparative Text Classification Using TF-IDF and Traditional Machine Learning
This module implements a complete, reproducible experimental pipeline for SMS
spam detection. It performs:
1. Robust dataset loading and data-quality validation.
2. Defensible duplicate-handling and class-distribution analysis.
3. TF-IDF feature extraction (unigrams + bigrams) fitted strictly on training data.
4. Training and evaluation of four machine learning classifiers:
- Decision Tree
- K-Nearest Neighbors (KNN)
- Support Vector Machine (LinearSVC)
- Multinomial Naive Bayes
5. Metric computation (Accuracy, Spam Precision, Spam Recall, Spam F1-Score).
6. Confusion matrix generation and comparative visualization.
7. What-if parameter sensitivity experiment (varying K in KNN: 3, 5, 7, 9).
8. Evidence-based model recommendation and trade-off analysis.
Dataset Source:
UCI Machine Learning Repository: SMS Spam Collection
DOI: https://doi.org/10.24432/C5VG8H
"""
import argparse
import os
import sys
import time
from pathlib import Path
from typing import Dict, List, Optional, Tuple, Any
import matplotlib
matplotlib.use("Agg") # Non-interactive backend for headless/script execution
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
Ln 1, Col 1
Spaces: 4
UTF-8
LF
Python
Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:28
28-10-2026

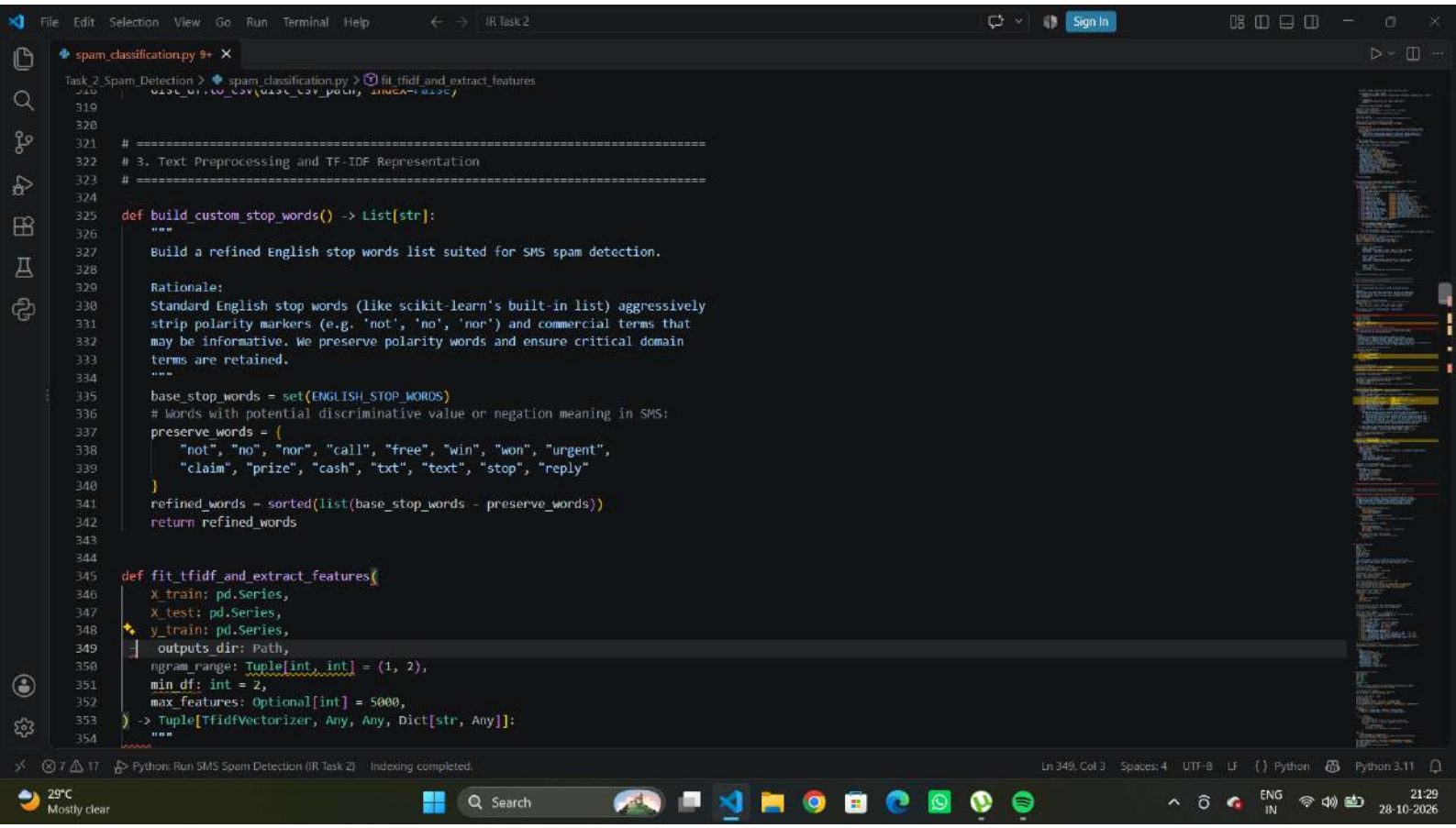

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > fit_tfidf_and_extract_features
def fit_tfidf_and_extract_features(
with open(feature_summary_path, "w", encoding="utf-8") as f:
f.write("=" * 78 + "\n")
f.write(" INFORMATION RETRIEVAL CLA2 – TASK 2: TF-IDF FEATURE SUMMARY\n")
f.write("=" * 78 + "\n\n")
f.write(f"Training Documents Count : {X_train.shape[0]:,}\n")
f.write(f"Testing Documents Count : {X_test.shape[0]:,}\n")
f.write(f"N-Gram Range : {ngram_range} (Unigrams + Bigrams)\n")
f.write(f"Minimum Document Frequency : {min_df}\n")
f.write(f"Maximum Features Ceiling : {max_features}\n")
f.write(f"Actual Vocabulary Size : {num_features:,} features\n")
f.write(f"Sublinear TF Scaling : True (1 + log(TF))\n")
f.write(f"Normalization : L2 Euclidean Norm\n\n")
f.write("TF-IDF WEIGHTING CONCEPT & INFORMATION RETRIEVAL FOUNDATION:\n")
f.write(
"TF-IDF (Term Frequency-Inverse Document Frequency) measures how important a word\n"
"or bigram is to a message within the corpus. It combines two factors:\n"
"1. Term Frequency (TF): Accounts for how many times a word occurs in a specific SMS.\n"
"2. Inverse Document Frequency (IDF): Decreases the weight of terms that appear in\n"
" almost every message (e.g. 'ok', 'good'), while heavily boosting words that appear\n"
" frequently only in specific contexts (e.g. 'claim', 'urgent', 'txt', 'prize').\n"
"This transformation yields a sparse vector representation where discriminative spam\n"
"indicators stand out distinctly from everyday interpersonal dialogue.\n\n"
)
f.write("SAMPLE OF TOP LEARNED TF-IDF FEATURES (CORPUS-WIDE AVERAGE WEIGHT):\n")
for rank, (feat, weight) in enumerate(top_features_sample, start=1):
f.write(f" {rank:2d}. {feat:<20} (mean TF-IDF weight: {weight:.5f})\n")
Generate Preprocessing Examples CSV (5 ham, 5 spam from training set)
analyzer = vectorizer.build_analyzer()
sample_rows = []
Pick 5 ham and 5 spam samples
ham_samples = X_train[y_train == "ham"].head(5)
spam_samples = X_train[y_train == "spam"].head(5)
combined_samples = pd.concat([ham_samples, spam_samples])
Python: Run SMS Spam Detection (IR Task 2) Indexing completed.
Ln 424, Col 21 Spaces: 4 UTF-8 LF Python Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026

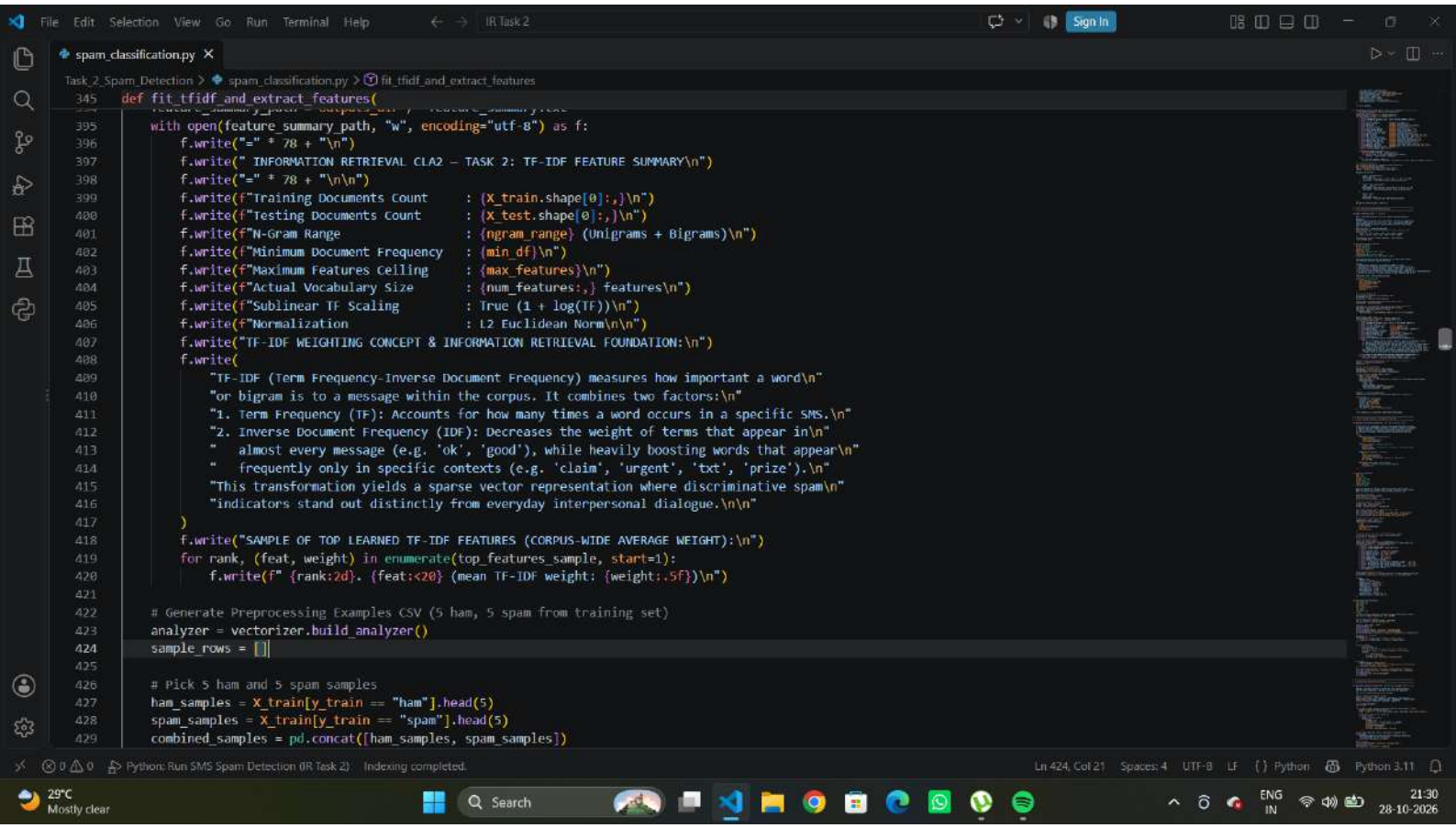

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > evaluate_single_model
def evaluate_single_model(
zero_division=0
)
Confusion Matrix: [[TN, FP], [FN, TP]] where ham=0, spam=1
cm = confusion_matrix(y_test, y_pred, labels=target_names)
tn, fp, fn, tp = cm.ravel()
Save Classification Report
slug = name.lower().replace(" ", "_").replace("-", "_")
report_file = outputs_dir / "classification_reports" / f"{slug}_report.txt"
with open(report_file, "w", encoding="utf-8") as f:
f.write("=" * 78 + "\n")
f.write(f" CLASSIFICATION REPORT: {name.upper()}\n")
f.write("=" * 78 + "\n\n")
f.write(f"Training Time : {t_train:.4f} seconds\n")
f.write(f"Prediction Time : {t_pred:.4f} seconds\n")
f.write(f"Overall Accuracy: {acc * 100:.2f}%\n")
f.write(f"Spam Precision : {prec:.4f}\n")
f.write(f"Spam Recall : {rec:.4f}\n")
f.write(f"Spam F1-Score : {f1:.4f}\n\n")
f.write("CONFUSION MATRIX BREAKDOWN:\n")
f.write(f" True Negatives (Ham correctly identified as Ham) : {tn:,}\n")
f.write(f" False Positives (Ham incorrectly flagged as Spam) : {fp:,}\n")
f.write(f" False Negatives (Spam missed, marked as Ham) : {fn:,}\n")
f.write(f" True Positives (Spam correctly identified as Spam) : {tp:,}\n\n")
f.write("DETAILED PER-CLASS METRICS:\n")
f.write(report_str)
Plot and Save Confusion Matrix
cm_plot_path = outputs_dir / "confusion_matrices" / f"{slug}_confusion_matrix.png"
plot_styled_confusion_matrix(cm, name, acc, prec, rec, f1, cm_plot_path)
return {
"Model": name,
"Accuracy": round(acc, 4),
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026

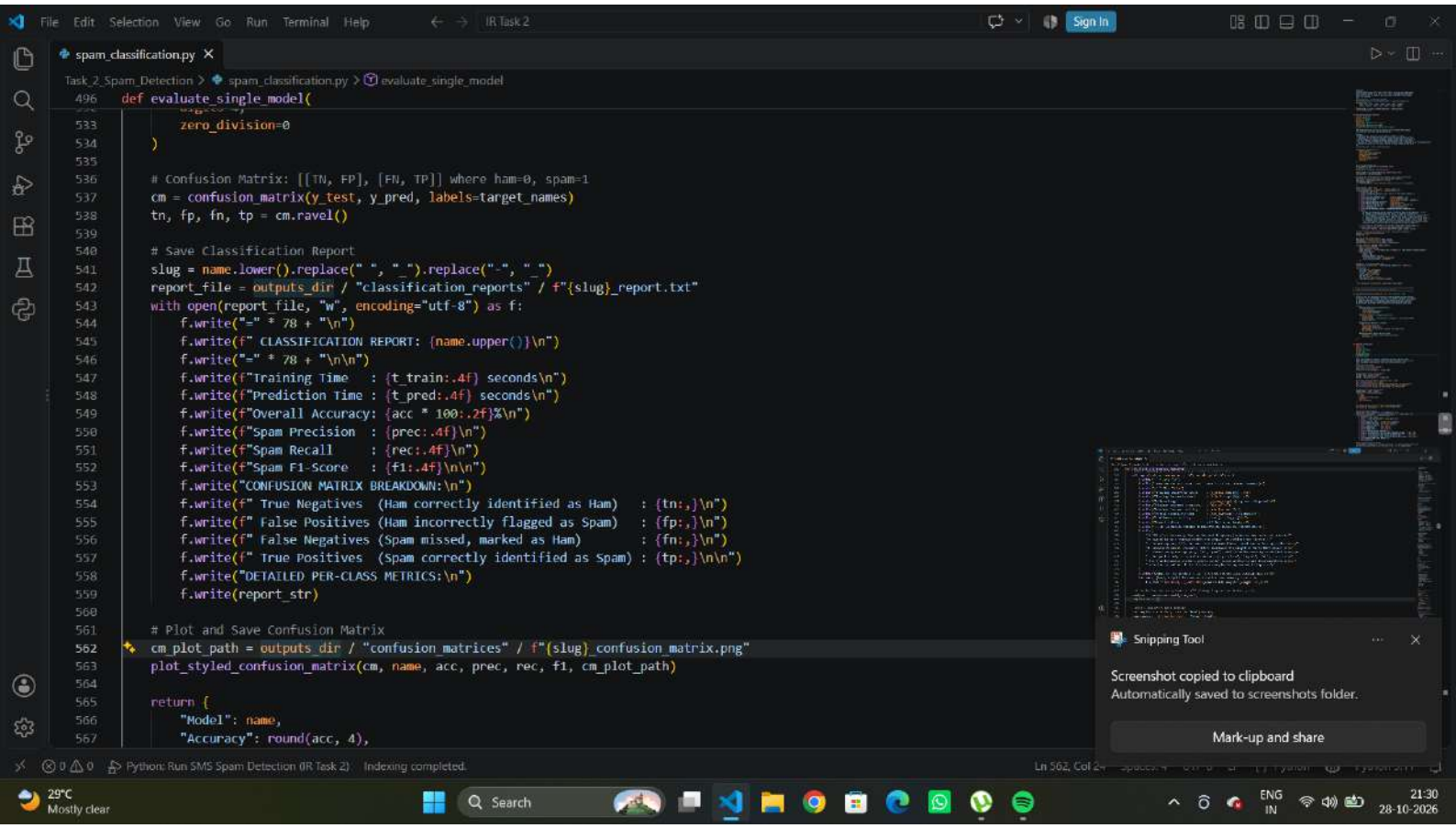

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > locate_dataset_file
==
2. Dataset Discovery, Loading, and Validation
==
def locate_dataset_file(explicit_path: Optional[str] = None, data_dir: Optional[Path] = None) -> Optional[Path]:
"""
Locate the SMS Spam Collection dataset file.
Searches explicit path, followed by common candidate names in the data directory.
"""
if explicit_path:
p = Path(explicit_path)
if p.is_file():
return p
print(f"[WARNING] Specified dataset path '{explicit_path}' was not found.")
if data_dir and data_dir.is_dir():
candidates = [
"SMSSpamCollection",
"SMSSpamCollection.txt",
"smsspamcollection",
"SMSSpamCollection.tsv",
"SMSSpamCollection.csv",
"sms_spam_collection.txt",
]
for name in candidates:
target = data_dir / name
if target.is_file() and target.stat().st_size > 0:
return target
Check for any non-hidden file in data_dir
for file in data_dir.iterdir():
if file.is_file() and not file.name.startswith(".") and not file.name.endswith(".zip"):
return file
return None
Python: Run SMS Spam Detection (IR Task 2) Indexing completed.
Ln 121, Col 1 Spaces: 4 UTF-8 LF Python Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:29
28-10-2026

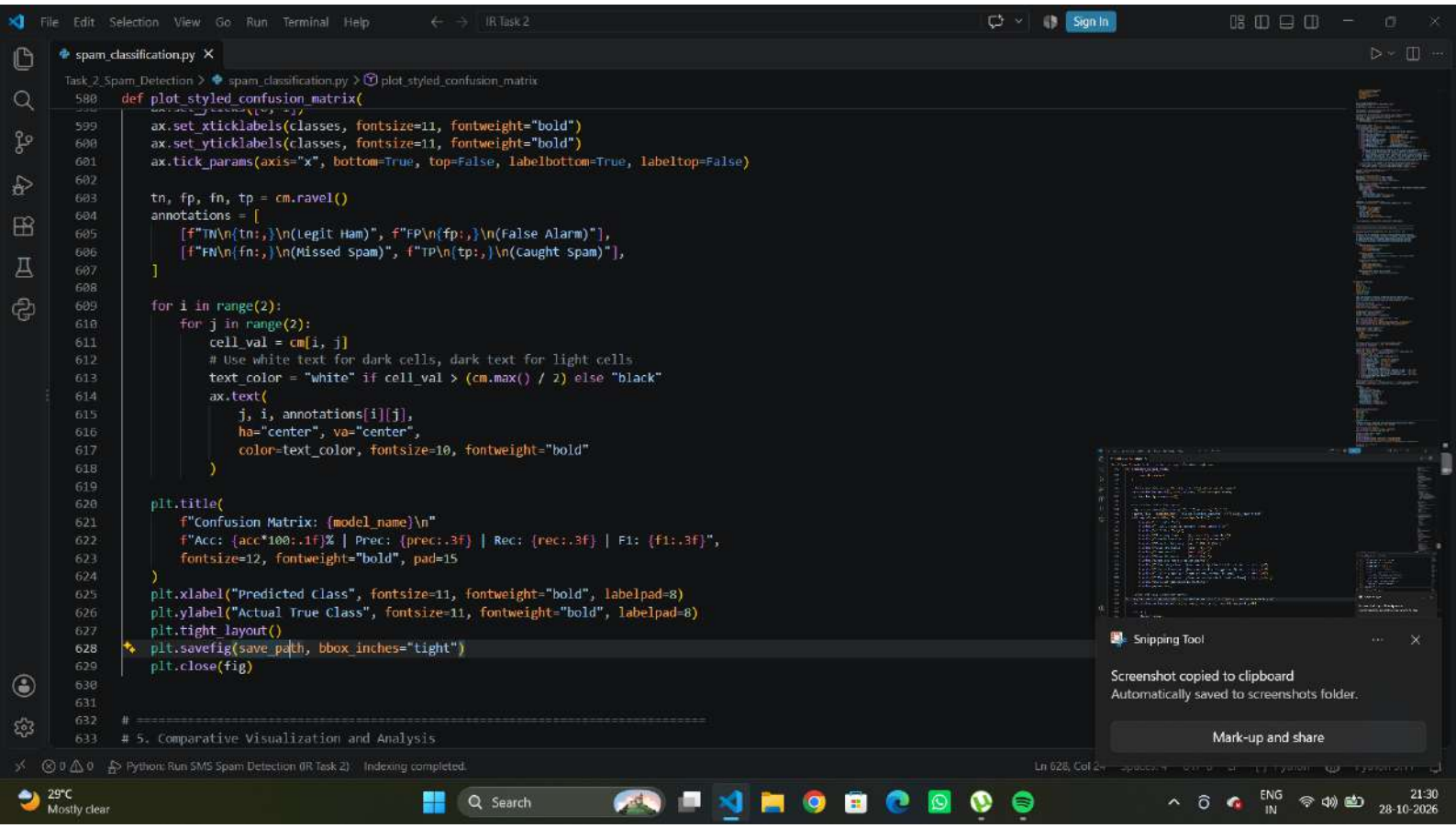

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > load_and_validate_dataset
def print_missing_dataset_help(data_dir: Path) -> None:
print(f"\n{banner}")
print(" [ERROR] SMS Spam Collection Dataset File Not Found")
print(f"{banner}")
print(" The required dataset could not be located in the data folder:")
print(f" Expected location: {data_dir.resolve() / 'SMSSpamCollection'}")
print("\n HOW TO FIX THIS IN 3 EASY STEPS:")
print(" 1. Visit the official UCI Machine Learning Repository:")
print(" https://doi.org/10.24432/C5VG8H")
print(" (or search 'UCI SMS Spam Collection')")
print(" 2. Download the archive ('smsspamcollection.zip').")
print(" 3. Extract the file named 'SMSSpamCollection' (no extension) directly into:")
print(f" {data_dir.resolve()}")
print("\n Once placed, re-run this script:")
print(" python spam_classification.py")
print(f"{banner}\n")
def load_and_validate_dataset(
file_path: Path,
remove_duplicates: bool = True
) -> Tuple[pd.DataFrame, Dict[str, Any]]:
"""
Load and validate the tab-separated SMS Spam Collection file.
Robust parsing:
- Reads line-by-line using UTF-8 (with latin-1 fallback if encoding issues occur).
- Splits strictly on the first tab character (tab-separated: label \\t message).
- Validates recognized labels ('ham', 'spam') and non-empty message bodies.
- Records malformed or skipped lines with reasons.
- Inspects and defensibly handles exact duplicates to prevent train-test data leakage.
"""
total_lines = 0
valid_records: List[Tuple[str, str]] = []
skipped_records: List[Tuple[int, str]] = []
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:29
28-10-2026

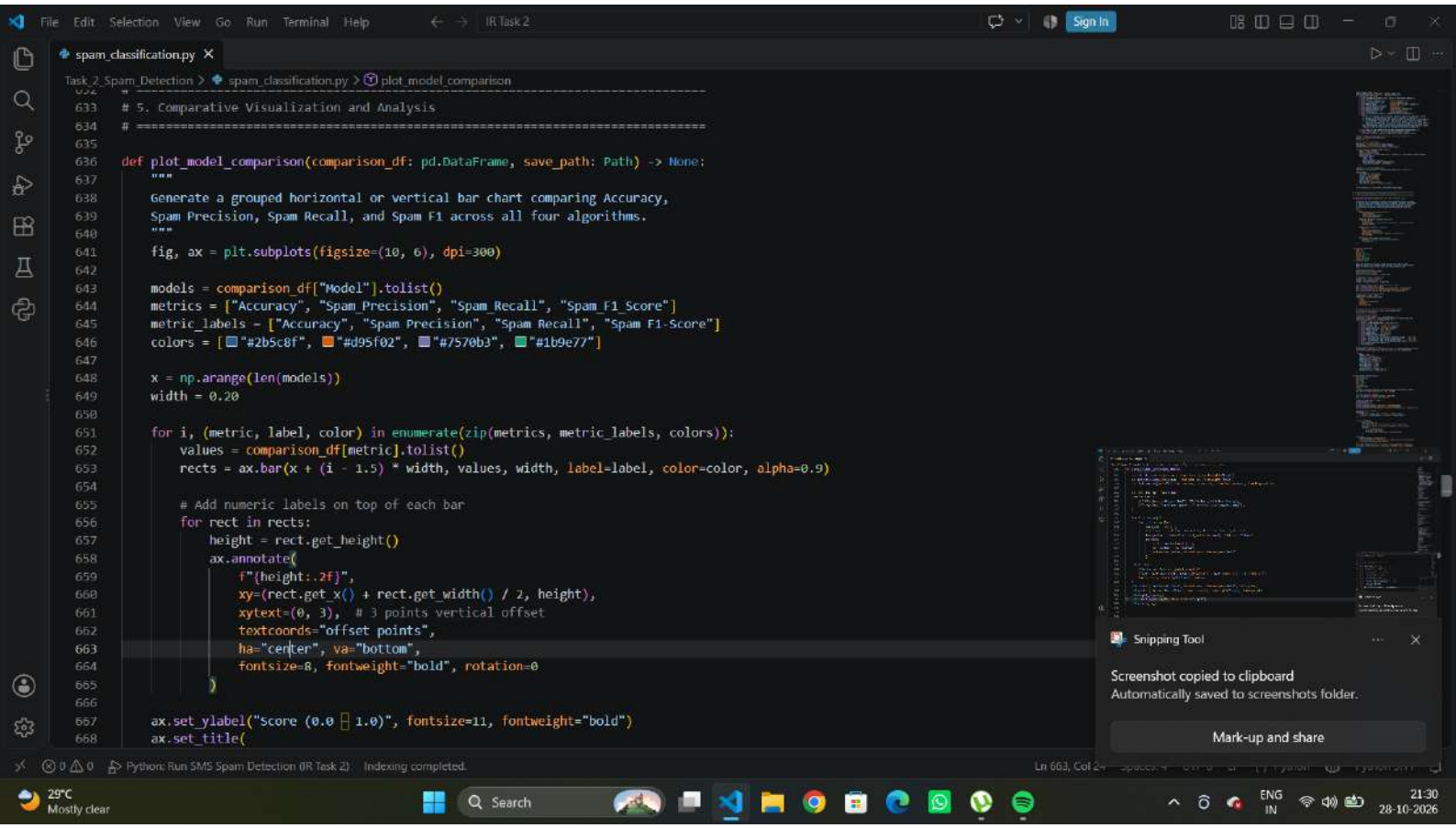

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > save_dataset_summary_files
def load_and_validate_dataset(
df = df_raw.copy()
dedup_reason = "Duplicates retained as requested by configuration."
final_label_counts = df["label"].value_counts().to_dict()
metadata: Dict[str, Any] = {
"file_name": file_path.name,
"file_path": str(file_path.resolve()),
"file_size_bytes": file_path.stat().st_size,
"encoding_used": chosen_encoding,
"total_lines_read": total_lines,
"valid_rows_parsed": raw_valid_count,
"skipped_rows_count": len(skipped_records),
"skipped_records_sample": skipped_records[:10],
"raw_label_counts": raw_label_counts,
"exact_duplicates_count": exact_duplicates_count,
"conflicting_messages_count": conflicting_messages_count,
"remove_duplicates_applied": remove_duplicates,
"dedup_reason": dedup_reason,
"retained_rows_count": len(df),
"final_label_counts": final_label_counts,
"missing_values_count": int(df.isnull().sum().sum()),
}
return df, metadata
def save_dataset_summary_files(metadata: Dict[str, Any], outputs_dir: Path) -> None:
"""Save dataset summary text and class distribution CSV."""
1. Dataset Summary Text
summary_txt_path = outputs_dir / "dataset_summary.txt"
with open(summary_txt_path, "w", encoding="utf-8") as f:
f.write("=" * 78 + "\n")
f.write(" INFORMATION RETRIEVAL CLA2 – TASK 2: DATASET SUMMARY & AUDIT\n")
f.write("=" * 78 + "\n\n")
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:29
28-10-2026

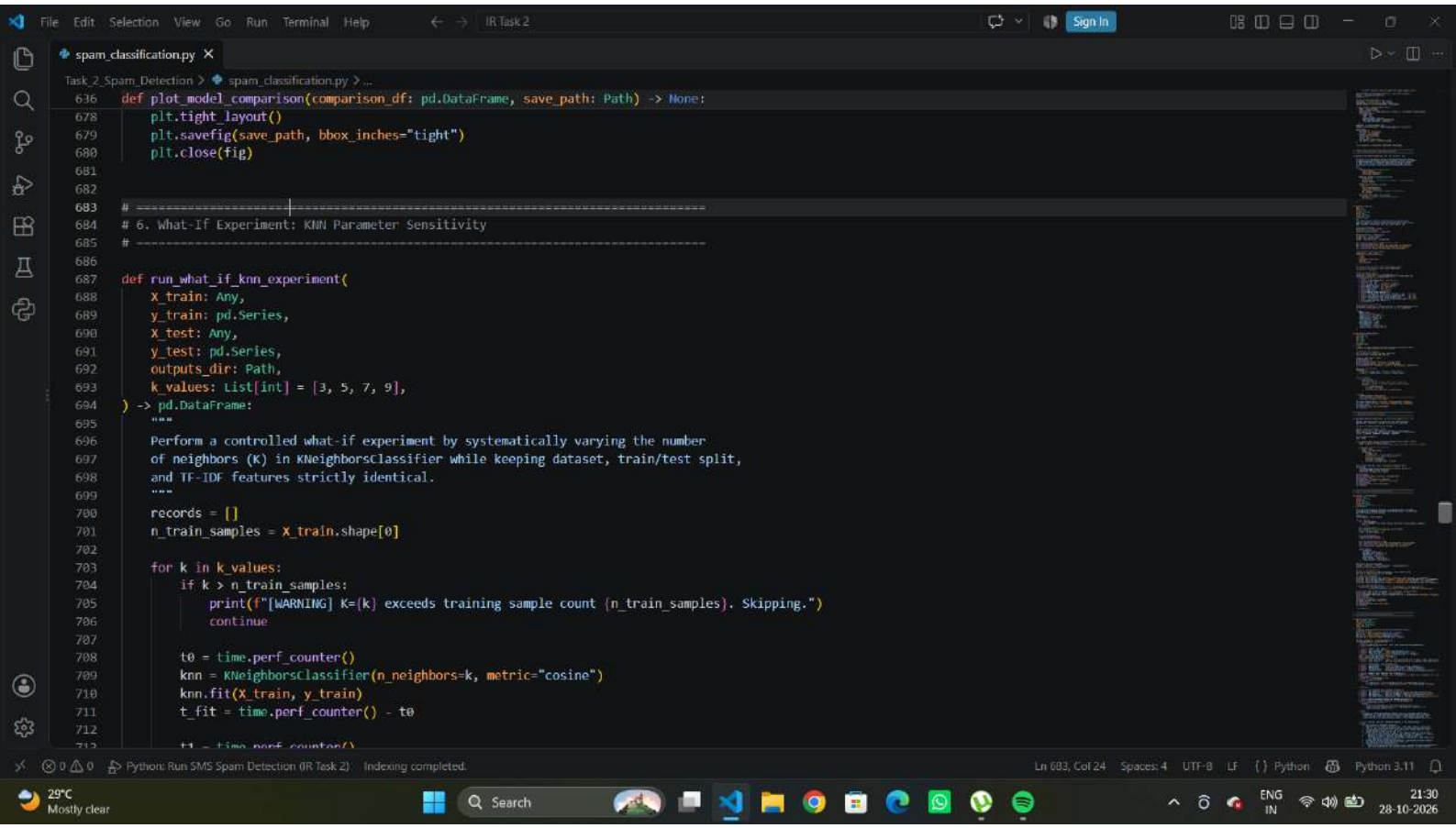

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py 9+
Task_2_Spam_Detection > spam_classification.py > fit_tfidf_and_extract_features
==========
3. Text Preprocessing and TF-IDF Representation
==========
def build_custom_stop_words() -> List[str]:
"""
Build a refined English stop words list suited for SMS spam detection.
Rationale:
Standard English stop words (like scikit-learn's built-in list) aggressively
strip polarity markers (e.g. 'not', 'no', 'nor') and commercial terms that
may be informative. We preserve polarity words and ensure critical domain
terms are retained.
"""
base_stop_words = set(ENGLISH_STOP_WORDS)
Words with potential discriminative value or negation meaning in SMS:
preserve_words = {
"not", "no", "nor", "call", "free", "win", "won", "urgent",
"claim", "prize", "cash", "txt", "text", "stop", "reply"
}
refined_words = sorted(list(base_stop_words - preserve_words))
return refined_words
def fit_tfidf_and_extract_features(
X_train: pd.Series,
X_test: pd.Series,
y_train: pd.Series,
outputs_dir: Path,
ngram_range: Tuple[int, int] = (1, 2),
min_df: int = 2,
max_features: Optional[int] = 5000,
) -> Tuple[TfidfVectorizer, Any, Any, Dict[str, Any]]:
"""
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
Ln 349, Col 3
Spaces: 4
UTF-8
LF
Python
Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:29
28-10-2026

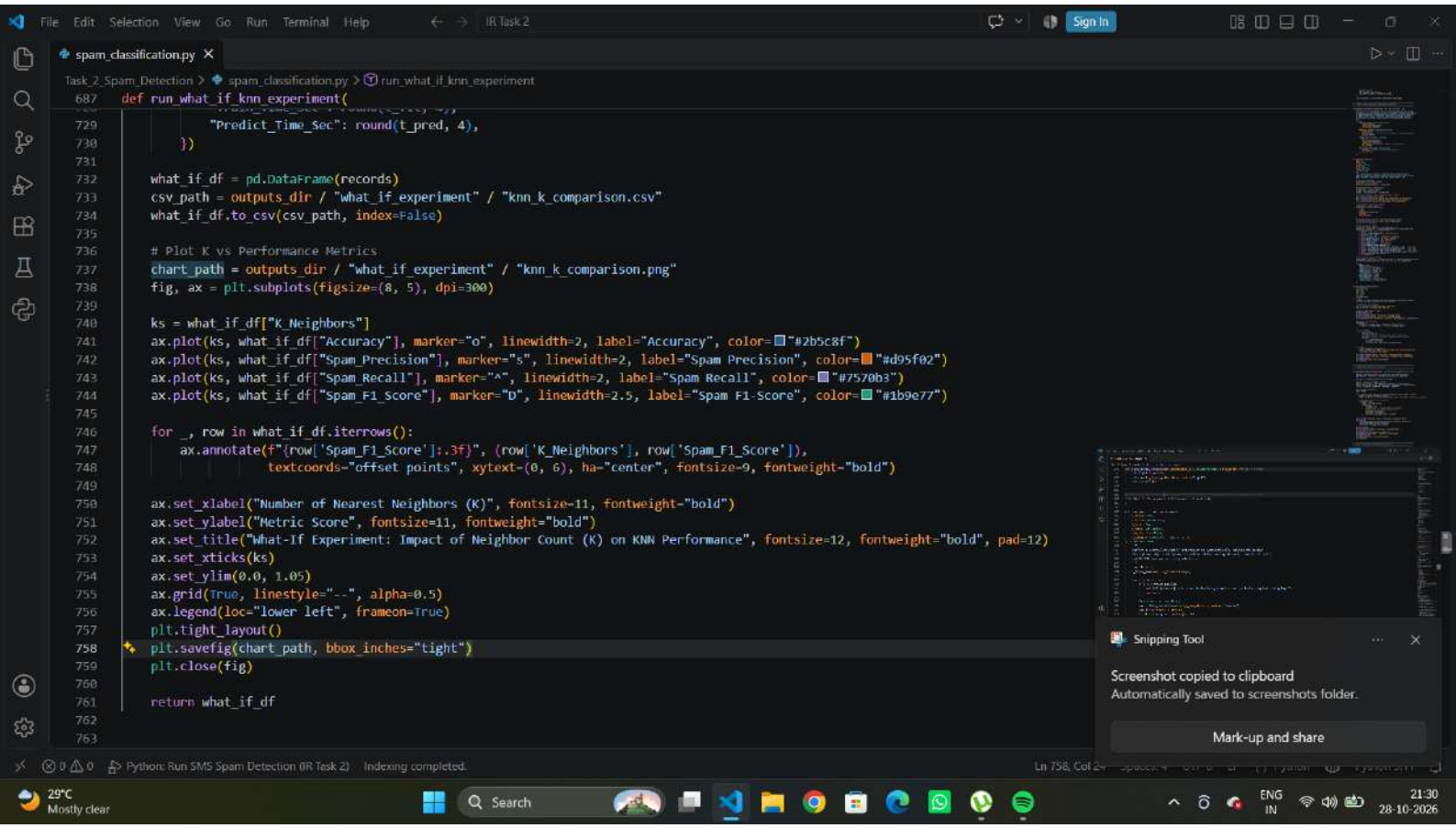

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > fit_tfidf_and_extract_features
def fit_tfidf_and_extract_features(
with open(feature_summary_path, "w", encoding="utf-8") as f:
f.write("=" * 78 + "\n")
f.write(" INFORMATION RETRIEVAL CLA2 – TASK 2: TF-IDF FEATURE SUMMARY\n")
f.write("=" * 78 + "\n\n")
f.write(f"Training Documents Count : {X_train.shape[0]:,}\n")
f.write(f"Testing Documents Count : {X_test.shape[0]:,}\n")
f.write(f"N-Gram Range : {ngram_range} (Unigrams + Bigrams)\n")
f.write(f"Minimum Document Frequency : {min_df}\n")
f.write(f"Maximum Features Ceiling : {max_features}\n")
f.write(f"Actual Vocabulary Size : {num_features:,} features\n")
f.write(f"Sublinear TF Scaling : True (1 + log(TF))\n")
f.write(f"Normalization : L2 Euclidean Norm\n\n")
f.write("TF-IDF WEIGHTING CONCEPT & INFORMATION RETRIEVAL FOUNDATION:\n")
f.write(
"TF-IDF (Term Frequency-Inverse Document Frequency) measures how important a word\n"
"or bigram is to a message within the corpus. It combines two factors:\n"
"1. Term Frequency (TF): Accounts for how many times a word occurs in a specific SMS.\n"
"2. Inverse Document Frequency (IDF): Decreases the weight of terms that appear in\n"
" almost every message (e.g. 'ok', 'good'), while heavily boosting words that appear\n"
" frequently only in specific contexts (e.g. 'claim', 'urgent', 'txt', 'prize').\n"
"This transformation yields a sparse vector representation where discriminative spam\n"
"indicators stand out distinctly from everyday interpersonal dialogue.\n\n"
)
f.write("SAMPLE OF TOP LEARNED TF-IDF FEATURES (CORPUS-WIDE AVERAGE WEIGHT):\n")
for rank, (feat, weight) in enumerate(top_features_sample, start=1):
f.write(f" {rank:2d}. {feat:<20} (mean TF-IDF weight: {weight:.5f})\n")
Generate Preprocessing Examples CSV (5 ham, 5 spam from training set)
analyzer = vectorizer.build_analyzer()
sample_rows = []
Pick 5 ham and 5 spam samples
ham_samples = X_train[y_train == "ham"].head(5)
spam_samples = X_train[y_train == "spam"].head(5)
combined_samples = pd.concat([ham_samples, spam_samples])
Python: Run SMS Spam Detection (IR Task 2) Indexing completed.
Ln 424, Col 21 Spaces: 4 UTF-8 LF Python Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026

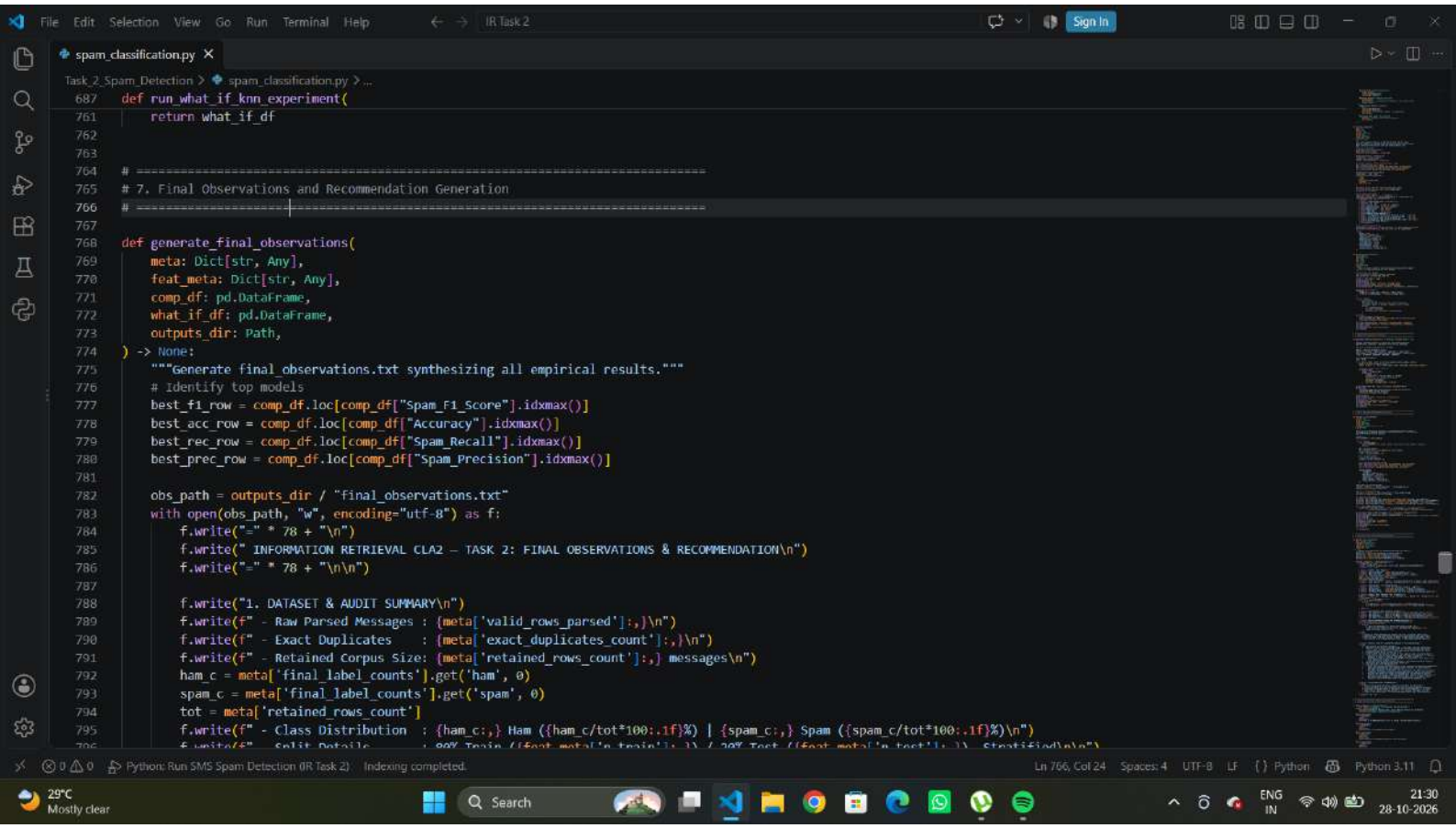

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > evaluate_single_model
def evaluate_single_model(
zero_division=0
)
Confusion Matrix: [[TN, FP], [FN, TP]] where ham=0, spam=1
cm = confusion_matrix(y_test, y_pred, labels=target_names)
tn, fp, fn, tp = cm.ravel()
Save Classification Report
slug = name.lower().replace(" ", "_").replace("-", "_")
report_file = outputs_dir / "classification_reports" / f"{slug}_report.txt"
with open(report_file, "w", encoding="utf-8") as f:
f.write("=" * 78 + "\n")
f.write(f" CLASSIFICATION REPORT: {name.upper()}\n")
f.write("=" * 78 + "\n\n")
f.write(f"Training Time : {t_train:.4f} seconds\n")
f.write(f"Prediction Time : {t_pred:.4f} seconds\n")
f.write(f"Overall Accuracy: {acc * 100:.2f}%\n")
f.write(f"Spam Precision : {prec:.4f}\n")
f.write(f"Spam Recall : {rec:.4f}\n")
f.write(f"Spam F1-Score : {f1:.4f}\n\n")
f.write("CONFUSION MATRIX BREAKDOWN:\n")
f.write(f" True Negatives (Ham correctly identified as Ham) : {tn:,}\n")
f.write(f" False Positives (Ham incorrectly flagged as Spam) : {fp:,}\n")
f.write(f" False Negatives (Spam missed, marked as Ham) : {fn:,}\n")
f.write(f" True Positives (Spam correctly identified as Spam) : {tp:,}\n\n")
f.write("DETAILED PER-CLASS METRICS:\n")
f.write(report_str)
Plot and Save Confusion Matrix
cm_plot_path = outputs_dir / "confusion_matrices" / f"{slug}_confusion_matrix.png"
plot_styled_confusion_matrix(cm, name, acc, prec, rec, f1, cm_plot_path)
return {
"Model": name,
"Accuracy": round(acc, 4),
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026

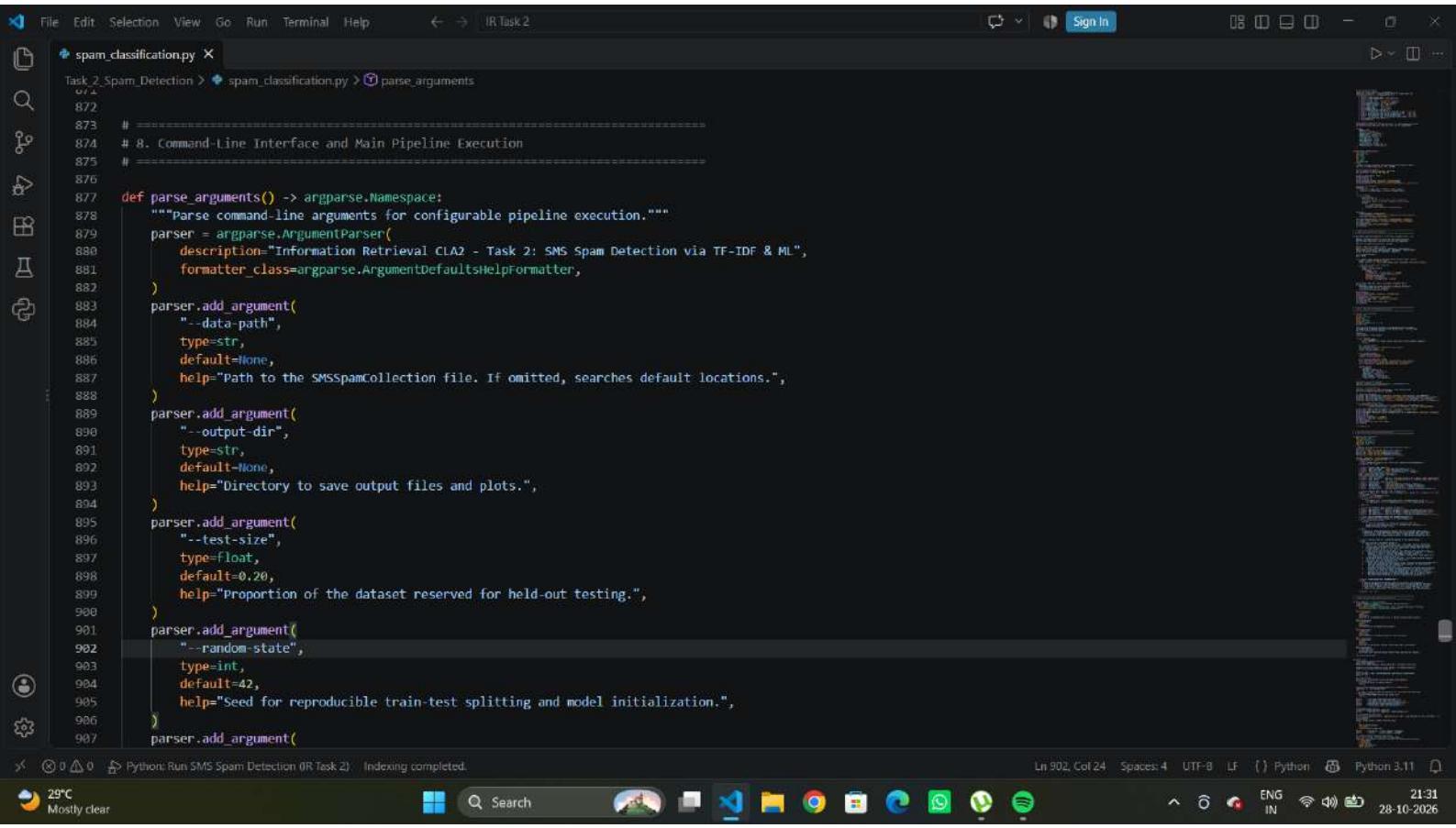

File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > plot_styled_confusion_matrix
def plot_styled_confusion_matrix(
ax.set_xticklabels(classes, fontsize=11, fontweight="bold")
ax.set_yticklabels(classes, fontsize=11, fontweight="bold")
ax.tick_params(axis="x", bottom=True, top=False, labelbottom=True, labeltop=False)
tn, fp, fn, tp = cm.ravel()
annotations = [
[f"TN\n{tn:,}\n(Legit Ham)", f"FP\n{fp:,}\n(False Alarm)"],
[f"FN\n{fn:,}\n(Missed Spam)", f"TP\n{tp:,}\n(Caught Spam)"],
]
for i in range(2):
for j in range(2):
cell_val = cm[i, j]
Use white text for dark cells, dark text for light cells
text_color = "white" if cell_val > (cm.max() / 2) else "black"
ax.text(
j, i, annotations[i][j],
ha="center", va="center",
color=text_color, fontsize=10, fontweight="bold"
)
plt.title(
f"Confusion Matrix: {model_name}\n"
f"Acc: {acc*100:.1f}% | Prec: {prec:.3f} | Rec: {rec:.3f} | F1: {f1:.3f}",
fontsize=12, fontweight="bold", pad=15
)
plt.xlabel("Predicted Class", fontsize=11, fontweight="bold", labelpad=8)
plt.ylabel("Actual True Class", fontsize=11, fontweight="bold", labelpad=8)
plt.tight_layout()
plt.savefig(save_path, bbox_inches="tight")
plt.close(fig)
==
5. Comparative Visualization and Analysis
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026


File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > plot_model_comparison
5. Comparative Visualization and Analysis
def plot_model_comparison(comparison_df: pd.DataFrame, save_path: Path) -> None:
"""
Generate a grouped horizontal or vertical bar chart comparing Accuracy,
Spam Precision, Spam Recall, and Spam F1 across all four algorithms.
"""
fig, ax = plt.subplots(figsize=(10, 6), dpi=300)
models = comparison_df["Model"].tolist()
metrics = ["Accuracy", "Spam Precision", "Spam Recall", "Spam F1_Score"]
metric_labels = ["Accuracy", "Spam Precision", "Spam Recall", "Spam F1-Score"]
colors = ["#2b5c8f", "#d95f02", "#7570b3", "#1b9e77"]
x = np.arange(len(models))
width = 0.20
for i, (metric, label, color) in enumerate(zip(metrics, metric_labels, colors)):
values = comparison_df[metric].tolist()
rects = ax.bar(x + (i - 1.5) * width, values, width, label=label, color=color, alpha=0.9)
Add numeric labels on top of each bar
for rect in rects:
height = rect.get_height()
ax.annotate(
f"{height:.2f}",
xy=(rect.get_x() + rect.get_width() / 2, height),
xytext=(0, 3), # 3 points vertical offset
textcoords="offset points",
ha="center", va="bottom",
fontsize=8, fontweight="bold", rotation=0
)
ax.set_ylabel("Score (0.0 - 1.0)", fontsize=11, fontweight="bold")
ax.set_title(
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026


File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > ...
636 def plot_model_comparison(comparison_df: pd.DataFrame, save_path: Path) -> None:
678 plt.tight_layout()
679 plt.savefig(save_path, bbox_inches="tight")
680 plt.close(fig)
681
682
683 # ==========
684 # 6. What-If Experiment: KNN Parameter Sensitivity
685 # ==========
686
687 def run_what_if_knn_experiment(
688 X_train: Any,
689 y_train: pd.Series,
690 X_test: Any,
691 y_test: pd.Series,
692 outputs_dir: Path,
693 k_values: List[int] = [3, 5, 7, 9],
694) -> pd.DataFrame:
695 """
696 Perform a controlled what-if experiment by systematically varying the number
697 of neighbors (K) in KNeighborsClassifier while keeping dataset, train/test split,
698 and TF-IDF features strictly identical.
699 """
700 records = []
701 n_train_samples = X_train.shape[0]
702
703 for k in k_values:
704 if k > n_train_samples:
705 print(f"[WARNING] K={k} exceeds training sample count ({n_train_samples}). Skipping.")
706 continue
707
708 t0 = time.perf_counter()
709 knn = KNeighborsClassifier(n_neighbors=k, metric="cosine")
710 knn.fit(X_train, y_train)
711 t_fit = time.perf_counter() - t0
712
Python: Run SMS Spam Detection (IR Task 2) Indexing completed.
Ln 683, Col 24 Spaces: 4 UTF-8 LF Python Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026


File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > run_what_if_knn_experiment
def run_what_if_knn_experiment(
"Predict_Time_Sec": round(t_pred, 4),
})
what_if_df = pd.DataFrame(records)
csv_path = outputs_dir / "what_if_experiment" / "knn_k_comparison.csv"
what_if_df.to_csv(csv_path, index=False)
Plot K vs Performance Metrics
chart_path = outputs_dir / "what_if_experiment" / "knn_k_comparison.png"
fig, ax = plt.subplots(figsize=(8, 5), dpi=300)
ks = what_if_df["K_Neighbors"]
ax.plot(ks, what_if_df["Accuracy"], marker="o", linewidth=2, label="Accuracy", color="#2b5c8f")
ax.plot(ks, what_if_df["Spam_Precision"], marker="s", linewidth=2, label="Spam Precision", color="#d95f02")
ax.plot(ks, what_if_df["Spam_Recall"], marker="^", linewidth=2, label="Spam Recall", color="#7570b3")
ax.plot(ks, what_if_df["Spam_F1_Score"], marker="D", linewidth=2.5, label="Spam F1-Score", color="#1b9e77")
for _, row in what_if_df.iterrows():
ax.annotate(f"{row['Spam_F1_Score']:.3f}", (row['K_Neighbors'], row['Spam_F1_Score']),
textcoords="offset points", xytext=(0, 6), ha="center", fontsize=9, fontweight="bold")
ax.set_xlabel("Number of Nearest Neighbors (K)", fontsize=11, fontweight="bold")
ax.set_ylabel("Metric Score", fontsize=11, fontweight="bold")
ax.set_title("What-If Experiment: Impact of Neighbor Count (K) on KNN Performance", fontsize=12, fontweight="bold", pad=12)
ax.set_xticks(ks)
ax.set_ylim(0.0, 1.05)
ax.grid(True, linestyle="--", alpha=0.5)
ax.legend(loc="lower left", frameon=True)
plt.tight_layout()
plt.savefig(chart_path, bbox_inches="tight")
plt.close(fig)
return what_if_df
Snipping Tool
Screenshot copied to clipboard
Automatically saved to screenshots folder.
Mark-up and share
Python: Run SMS Spam Detection (IR Task 2)
Indexing completed.
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026


File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > ...
687 def run_what_if_knn_experiment(
761 return what_if_df
762
763
764 # ==
765 # 7. Final Observations and Recommendation Generation
766 # ==
767
768 def generate_final_observations(
769 meta: Dict[str, Any],
770 feat_meta: Dict[str, Any],
771 comp_df: pd.DataFrame,
772 what_if_df: pd.DataFrame,
773 outputs_dir: Path,
774) -> None:
775 """Generate final_observations.txt synthesizing all empirical results."""
776 # Identify top models
777 best_f1_row = comp_df.loc[comp_df["Spam_F1_Score"].idxmax()]
778 best_acc_row = comp_df.loc[comp_df["Accuracy"].idxmax()]
779 best_rec_row = comp_df.loc[comp_df["Spam_Recall"].idxmax()]
780 best_prec_row = comp_df.loc[comp_df["Spam_Precision"].idxmax()]
781
782 obs_path = outputs_dir / "final_observations.txt"
783 with open(obs_path, "w", encoding="utf-8") as f:
784 f.write("=" * 78 + "\n")
785 f.write(" INFORMATION RETRIEVAL CLA2 – TASK 2: FINAL OBSERVATIONS & RECOMMENDATION\n")
786 f.write("=" * 78 + "\n\n")
787
788 f.write("1. DATASET & AUDIT SUMMARY\n")
789 f.write(f" - Raw Parsed Messages : {meta['valid_rows_parsed']:,}\n")
790 f.write(f" - Exact Duplicates : {meta['exact_duplicates_count']:,}\n")
791 f.write(f" - Retained Corpus Size: {meta['retained_rows_count']:,} messages\n")
792 ham_c = meta['final_label_counts'].get('ham', 0)
793 spam_c = meta['final_label_counts'].get('spam', 0)
794 tot = meta['retained_rows_count']
795 f.write(f" - Class Distribution : {ham_c:,} Ham ({ham_c/tot*100:.1f}%) | {spam_c:,} Spam ({spam_c/tot*100:.1f}%)\n")
Python: Run SMS Spam Detection (IR Task 2) Indexing completed.
Ln 766, Col 24 Spaces: 4 UTF-8 LF { } Python Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:30
28-10-2026


File Edit Selection View Go Run Terminal Help
IR Task 2
Sign In
spam_classification.py
Task_2_Spam_Detection > spam_classification.py > parse_arguments
==
8. Command-Line Interface and Main Pipeline Execution
==
def parse_arguments() -> argparse.Namespace:
"""Parse command-line arguments for configurable pipeline execution."""
parser = argparse.ArgumentParser(
description="Information Retrieval CLA2 - Task 2: SMS Spam Detection via TF-IDF & ML",
formatter_class=argparse.ArgumentDefaultsHelpFormatter,
)
parser.add_argument(
"--data-path",
type=str,
default=None,
help="Path to the SMSSpamCollection file. If omitted, searches default locations.",
)
parser.add_argument(
"--output-dir",
type=str,
default=None,
help="Directory to save output files and plots.",
)
parser.add_argument(
"--test-size",
type=float,
default=0.20,
help="Proportion of the dataset reserved for held-out testing.",
)
parser.add_argument(
"--random-state",
type=int,
default=42,
help="Seed for reproducible train-test splitting and model initialization.",
)
parser.add_argument(
Python: Run SMS Spam Detection (IR Task 2) Indexing completed.
Ln 902, Col 24 Spaces: 4 UTF-8 LF Python Python 3.11
29°C
Mostly clear
Search
ENG
IN
21:31
28-10-2026